

Streszczenie

Niniejsza praca przedstawia implementację aplikacji webowej społeczności wspinaczkowej. System została zaprojektowana i zrealizowana w języku JavaScript z wykorzystaniem biblioteki React po stronie klienta oraz środowiska Node.js z frameworkiem Express po stronie serwera. Do obsługi warstwy danych zastosowano bibliotekę Sequelize oraz bazę danych MySQL. Głównym celem aplikacji jest centralizacja informacji oraz usług związanych z tą dyscypliną sportu. Opracowany system umożliwia użytkownikom przeglądanie aktualności, listy wydarzeń oraz realizację zakupów online w jednym, spójnym środowisku internetowym. Poprawność działania aplikacji została zweryfikowana poprzez przeprowadzenie automatycznych testów z użyciem narzędzia Cypress i Jest.

Słowa kluczowe

wspinaczka sportowa, e-commerce, aplikacja webowa, JavaScript, React, Express, Mysql, Cypress, Jest.

Spis treści

Streszczenie	1
Słowa kluczowe	1
Wyraz Skrótów.....	4
Wstęp	5
1. Aplikacja webowa dla społeczności wspinaczkowej	6
1.1. Analiza istniejących rozwiązań	6
1.2. Cele i zastosowanie aplikacji	7
1.3. Elementy systemu i zakres funkcjonalności.....	8
1.2.1. Użytkownik indywidualny.....	8
1.2.2. Konto firmowe	8
1.2.3. Konto pracownika / administratora	9
2. Architektura aplikacji webowych	10
2.1. Model „klient-serwer”	10
2.2. Aplikacja typu Single Page Application.....	10
2.3. Warstwa frontendowa aplikacji webowej	11
2.4. Komunikacja frontend–backend	13
2.5. Wspinaczkowa aplikacja webowa w kontekście architektury klient–serwer.....	13
3. Wykorzystane technologie.....	16
3.1. Język JavaScript.....	16
3.2. React.....	16
3.3. CSS.....	18
3.4. i18next	18
3.5. Node.js	19
3.6. Express	20
3.7. Relacyjna baza danych MySQL.....	21
3.8. Sequelize	21
3.9. Cypress.....	26
3.10. JestJS.....	27
4. Implementacja	29
4.1. Diagram klas	29

4.2. Mechanizm uwierzytelniania i autoryzacji użytkowników	31
4.3. Mechanizm kontroli dostępu w warstwie frontendowej	32
4.4. Moduł zakupów i sprzedaży	32
4.5. Moduł wydarzeń i turniejów	35
4.6. Moduł artykułów i aktualności	37
4.7. Panel administracyjny i zarządzanie systemem	39
5. Testowanie aplikacji	41
5.1. Testy API i testy integracyjne backendu za pomocą JestJS.....	42
5.2. Testy end-to-end aplikacji z wykorzystaniem Cypress	59
6. Możliwości rozwoju aplikacji.....	70
6.1. Rozwój i optymalizacja zastosowanych technologii.....	70
6.2. Rozszerzenie interakcji między użytkownikami	71
6.3. Wdrożenie aplikacji mobilnej.....	72
6.4. Moduł treningowy	72
7. Podsumowanie.....	73
Bibliografia.....	75
Spis rysunków.....	77
Spis tabel.....	79

Wyraz Skrótów

HTTP - Hypertext Transfer Protocol

SPA - Single Page Application

SEO - Search Engine Optimization

GUI - Graphical user interface

DOM - Document Object Model

HTML - HyperText Markup Language

CSS - Cascading Style Sheets

JSON - JavaScript Object Notation

JWT – JSON Web Token

RDBMS - Relational Database Management System

SQL - Structured Query Language

ORM - Object-Relational Mapping

E2E – End-to-end

WCAG - Web Content Accessibility Guidelines

Wstęp

W ostatnich latach można zaobserwować dynamiczny rozwój aplikacji internetowych wspierających różne dziedziny życia codziennego, w tym również aktywność fizyczną i sport. W przypadku sportów masowych, takich jak piłka nożna, użytkownicy mają dostęp do rozbudowanych platform informacyjnych, serwisów społecznościowych oraz sklepów internetowych, które umożliwiają szybki dostęp do aktualności, wydarzeń oraz produktów związanych z daną dyscypliną.

W przeciwieństwie do sportów masowych, wspinaczka sportowa nadal pozostaje dyscypliną niszową. Mimo rosnącej popularności ścian wspinaczkowych oraz sukcesów polskich zawodników na arenie międzynarodowej, środowisko wspinaczkowe nie dysponuje jednolitym, powszechnie stosowanym narzędziem informatycznym umożliwiającym kompleksowe zarządzanie informacjami związanymi z tą aktywnością. Informacje istotne dla środowiska wspinaczkowego, takie jak wydarzenia sportowe, zawody, warsztaty, nowości sprzętowe czy aktualności branżowe są często rozproszone pomiędzy wiele niezależnych serwisów internetowych. Brak jednego, scentralizowanego miejsca utrudnia użytkownikom szybkie i wygodne dotarcie do potrzebnych informacji.

Celem niniejszej pracy jest zaprojektowanie oraz implementacja aplikacji webowej dedykowanej społeczności wspinaczkowej, której zadaniem jest centralizacja informacji oraz usług związanych z tą dyscypliną sportu. Opracowany system umożliwia użytkownikom indywidualnym przeglądanie aktualności, listy wydarzeń oraz realizację zakupów online w jednym, spójnym środowisku internetowym. Aplikacja uwzględnia również potrzeby podmiotów komercyjnych, takich jak producenci sprzętu wspinaczkowego czy organizatorzy wydarzeń, którzy mogą prezentować swoją ofertę oraz publikować informacje o organizowanych wydarzeniach.

W dalszej części pracy przedstawiono wykorzystane technologie, proces projektowania systemu, implementację kluczowych modułów aplikacji oraz sposób weryfikacji poprawności działania za pomocą testów.

1. Aplikacja webowa dla społeczności wspinaczkowej

1.1. Analiza istniejących rozwiązań

Większość obecnie funkcjonujących serwisów internetowych związanych ze wspinaczką sportową jest powiązana z konkretną ścianką wspinaczkową, klubem sportowym lub siecią obiektów. W konsekwencji publikowane w nich informacje, takie jak turnieje, wydarzenia czy aktualności, mają najczęściej charakter lokalny i ograniczają się do jednego regionu lub organizatora.

Projektowana aplikacja webowa została zaprojektowana jako rozwiązanie o szerszym zasięgu, niezależne od pojedynczego obiektu. Umożliwia ona centralne gromadzenie wydarzeń, informacji oraz ofert pochodzących od różnych podmiotów, co pozwala użytkownikom na dostęp do bardziej zróżnicowanych i aktualnych treści w jednym miejscu.

Istotną różnicą w porównaniu do wielu istniejących systemów jest również rozszerzony model użytkowników. Oprócz standardowego podziału na użytkownika i administratora, aplikacja uwzględnia rolę kont firmowych, które mogą publikować własne produkty, wydarzenia lub treści, co pozwala na bezpośrednią interakcję między użytkownikami a podmiotami komercyjnymi.

Na rynku dostępne są również rozbudowane, międzynarodowe portale tematyczne poświęcone wspinaczce, które oferują bogate bazy danych dróg wspinaczkowych, artykuły, oraz fora dyskusyjne. Serwisy tego typu pełnią istotną rolę informacyjną i społecznościową, jednak ich funkcjonalność koncentruje się głównie na aspektach sportowych i dokumentacyjnych.

W porównaniu do tego typu rozwiązań, projektowana aplikacja webowa kładzie większy nacisk na bieżące wydarzenia, turnieje oraz bezpośrednie interakcje pomiędzy użytkownikami a podmiotami komercyjnymi, takimi jak firmy oferujące produkty lub usługi związane ze wspinaczką. Aplikacja pełni rolę narzędzia integrującego aktualności, wydarzenia oraz procesy zakupowe w jednym spójnym środowisku.

Do istniejących rozwiązań należą również portale sprzedażowe i sklepy internetowe oferujące sprzęt wspinaczkowy. Systemy tego typu koncentrują się przede wszystkim na procesie sprzedaży, prezentacji oferty produktowej oraz obsłudze transakcji. Funkcjonalność informacyjna i społecznościowa jest w nich zazwyczaj ograniczona do podstawowych opisów produktów, opinii klientów lub komunikatów marketingowych.

W odróżnieniu od klasycznych portali e-commerce, projektowana aplikacja webowa nie pełni wyłącznie roli sklepu internetowego. Proces zakupowy stanowi jeden z modułów systemu, który funkcjonuje równolegle z modułami aktualności oraz wydarzeń sportowych. Dodatkową różnicą jest sposób prezentacji podmiotów komercyjnych. W projektowanym systemie firmy posiadają dedykowane konta umożliwiające nie tylko sprzedaż produktów, ale również publikowanie informacji oraz wydarzeń.

Analiza istniejących rozwiązań pokazuje, że obecnie dostępne systemy wspinaczkowe są najczęściej wyspecjalizowane w jednym obszarze, takim jak informacja lokalna, dokumentacja sportowa lub sprzedaż internetowa. Brakuje jednak narzędzia, które w sposób spójny integrowałoby funkcje informacyjne, społecznościowe oraz handlowe w jednym środowisku. Rozproszenie treści oraz ograniczony zakres interakcji pomiędzy użytkownikami i podmiotami komercyjnymi utrudniają wygodne korzystanie z zasobów branży wspinaczkowej. Wskazuje to na potrzebę stworzenia jednej platformy integrującej kluczowe funkcjonalności w sposób spójny i dostępny dla różnych grup użytkowników.

1.2. Cele i zastosowanie aplikacji

Głównym założeniem projektowanej aplikacji jest stworzenie scentralizowanego serwisu internetowego odpowiadającego na potrzeby informacyjne oraz użytkowe społeczności wspinaczkowej. System ma ułatwiać dostęp do aktualnych informacji, takich jak wydarzenia, nowości branżowe oraz oferta produktowa, które dotychczas były rozproszone pomiędzy wiele niezależnych źródeł.

Istotnym aspektem funkcjonowania aplikacji jest również zapewnienie przestrzeni dla podmiotów komercyjnych, w tym producentów sprzętu wspinaczkowego oraz organizatorów wydarzeń, umożliwiającej prezentowanie oferty oraz publikowanie treści w jednym, spójnym środowisku. W efekcie aplikacja pełni rolę platformy pośredniczącej pomiędzy użytkownikami końcowymi a firmami działającymi w branży wspinaczkowej, wspierając wymianę informacji i interakcję pomiędzy tymi grupami.

Zastosowanie aplikacji obejmuje zarówno codzienne przeglądanie informacji przez użytkowników indywidualnych, jak i wykorzystanie systemu przez konta firmowe do publikowania treści oraz ofert. System może być wykorzystywany jako narzędzie informacyjne, promocyjne oraz sprzedażowe, dostępne z poziomu dowolnego urządzenia wyposażonego w przeglądarkę internetową.

1.3. Elementy systemu i zakres funkcjonalności

Projektowana aplikacja webowa składa się z trzech podstawowych warstw: warstwy frontendowej odpowiedzialnej za interfejs użytkownika, warstwy backendowej realizującej logikę biznesową oraz warstwy danych przechowującej informacje o użytkownikach i zasobach aplikacji. Takie podejście umożliwia czytelny podział odpowiedzialności pomiędzy poszczególne elementy systemu oraz ułatwia jego dalszy rozwój i utrzymanie.

System przewiduje obsługę trzech typów użytkowników, różniących się zakresem dostępnych funkcjonalności oraz poziomem uprawnień.

1.2.1. Użytkownik indywidualny

Użytkownik indywidualny korzysta z aplikacji głównie w celu przeglądania treści oraz realizacji zakupów. Do podstawowych funkcjonalności dostępnych dla tego typu użytkownika należą.

- przeglądanie aktualności związanych ze wspinaczką sportową,
- skomentowanie artykułu,
- wyświetlanie listy wydarzeń oraz szczegółowych informacji o wydarzeniach,
- przeglądanie oferty produktowej dostępnej w systemie,
- realizacja zakupów online i dodawanie recenzji,
- zarządzanie podstawowymi danymi swojego konta użytkownika.

1.2.2. Konto firmowe

Konto firmowe przeznaczone jest dla podmiotów komercyjnych działających w branży wspinaczkowej, takich jak producenci sprzętu czy organizatorzy wydarzeń. Funkcjonalności dostępne dla kont firmowych obejmują:

- dodawanie, edytowanie oraz usuwanie ofert produktowych,
- publikowanie oraz zarządzanie wydarzeniami organizowanymi przez daną firmę,
- przeglądanie informacji dotyczących własnych produktów i wydarzeń,
- przeglądanie i zarządzanie zamówieniami,
- zarządzanie danymi firmowymi widocznymi w systemie.

1.2.3. Konto pracownika / administratora

Konto pracownika lub administratora systemu posiada rozszerzone uprawnienia, umożliwiające zarządzanie całością aplikacji oraz jej zawartością. Do funkcjonalności dostępnych dla pracownika należą:

- publikowanie i edytowanie aktualności dotyczących wspinaczki sportowej,
- moderowanie treści dodawanych przez użytkowników oraz konta firmowe,
- zarządzanie użytkownikami systemu oraz przypisywanie ról,
- przeglądanie danych dotyczących klientów, produktów oraz realizowanych zakupów,
- nadzorowanie poprawności działania systemu oraz jego bezpieczeństwa.

Dodatkowo administrator może zarządzać kontami pracowników, obejmujące tworzenie, edycję oraz dezaktywację kont.

2. Architektura aplikacji webowych

2.1. Model „klient-serwer”

Model klient–serwer jest fundamentalną architekturą stosowaną w aplikacjach webowych, w tym w projektowanej aplikacji dla społeczności wspinaczkowej. Architektura ta opiera się na rozdzieleniu odpowiedzialności pomiędzy komponenty systemu: klienta oraz serwera [1].

Zgodnie z dokumentacją IBM, w architekturze klient–serwer aplikacje klienckie mogą działać na tym samym komputerze co serwer bazy danych lub na innym urządzeniu w sieci. Klient wysyła żądania dostępu do usług i danych, a serwer odpowiada, udostępniając odpowiednie informacje lub wykonując zlecone operacje. Proces ten realizowany jest za pośrednictwem protokołów sieciowych i interfejsów komunikacyjnych, które umożliwiają przesyłanie danych pomiędzy klientem a serwerem [2].

W kontekście aplikacji webowej klientem jest najczęściej przeglądarka internetowa, w której działa interfejs użytkownika. Klient ten wysyła do serwera żądania HTTP, na przykład o pobranie listy wydarzeń lub dokumentów produktowych. Serwer aplikacji, odbiera te żądania, przetwarza je i komunikuje się z warstwą danych, po czym odsyła odpowiedź do klienta. Taki model działania realizuje zasadę pytanie–odpowiedź typową dla komunikacji w sieci Internet [1].

Architektura klient–serwer charakteryzuje się tym, że:

- klient jest odpowiedzialny za prezentację danych oraz interakcję z użytkownikiem,
- serwer realizuje logikę biznesową oraz zarządza dostępem do danych i zasobów,
- komunikacja między klientem a serwerem odbywa się za pomocą standardowych protokołów sieciowych (np. HTTP/HTTPS) [3].

W modelu tym możliwe jest także równoczesne obsługiwane wielu klientów przez jeden serwer — serwer odpowiada na każde żądanie niezależnie, co umożliwia skalowanie aplikacji i obsługę wielu użytkowników jednocześnie. Taki podział odpowiedzialności ułatwia również rozdzielenie warstw systemu oraz ułatwia jego rozwój i utrzymanie w przyszłości [1].

2.2. Aplikacja typu Single Page Application

Aplikacja typu Single Page Application (SPA) jest implementacją aplikacji webowej, w której podczas pierwszego załadowania pobierany jest jeden dokument HTML, a następnie zawartość tego dokumentu jest dynamicznie aktualizowana po stronie klienta. Zmiany treści

realizowane są przy użyciu języka JavaScript oraz interfejsów programistycznych, takich jak Fetch API, bez konieczności ponownego ładowania całej strony z serwera [4].

Umożliwia to użytkownikom korzystanie z aplikacji webowej w sposób bardziej płynny, ponieważ przechodzenie pomiędzy poszczególnymi widokami nie wymaga każdorazowego pobierania nowych dokumentów HTML. Zamiast tego aplikacja komunikuje się z serwerem w celu pobrania wyłącznie niezbędnych danych, które następnie są renderowane po stronie klienta. W konsekwencji możliwe jest ograniczenie liczby zapytań sieciowych oraz zmniejszenie ilości przesyłanych danych [4].

Jedną z głównych zalet aplikacji typu SPA jest zwiększona wydajność oraz responsywność interfejsu użytkownika, co przekłada się na bardziej dynamiczne i zbliżone do aplikacji desktopowych doświadczenie użytkownika [4].

Zastosowanie architektury SPA wiąże się jednak również z pewnymi ograniczeniami. Do najczęściej wskazywanych należą wyzwania związane z optymalizacją pod kątem wyszukiwarek internetowych (SEO), ponieważ treść strony jest generowana dynamicznie po stronie klienta. Dodatkowo implementacja aplikacji SPA wymaga bardziej zaawansowanego zarządzania stanem aplikacji, obsługi nawigacji oraz mechanizmów monitorowania wydajności. W porównaniu do tradycyjnych aplikacji wielostronicowych, zwiększa to złożoność projektu oraz nakład pracy potrzebny na jego utrzymanie [4].

Pomimo wskazanych ograniczeń, architektura SPA jest obecnie powszechnie stosowana w nowoczesnych aplikacjach webowych, zwłaszcza w systemach wymagających częstej interakcji z użytkownikiem oraz dynamicznej aktualizacji danych. Z tego względu stanowi ona odpowiednie rozwiązanie dla projektowanej aplikacji webowej, w której kluczowe znaczenie ma wygodny dostęp do aktualności, wydarzeń oraz funkcjonalności zakupowych [5].

2.3. Warstwa frontendowa aplikacji webowej

Pojęcie frontend odnosi się do graficznego interfejsu użytkownika (GUI), z którym użytkownik końcowy ma bezpośrednią interakcję. Obejmuje on elementy takie jak menu nawigacyjne, przyciski, obrazy, formularze, wykresy oraz inne komponenty interfejsu użytkownika. Z technicznego punktu widzenia ekran lub strona widoczna dla użytkownika składa się z wielu elementów interfejsu, które tworzą model obiektowy dokumentu (Document Object Model, DOM). DOM stanowi hierarchiczną reprezentację struktury strony internetowej i umożliwia jej dynamiczną modyfikację [6].

Interakcja użytkownika z warstwą frontendową aplikacji webowej opiera się na trzech podstawowych technologiach, które definiują sposób prezentacji oraz zachowania interfejsu:

- HTML (HyperText Markup Language) odpowiada za strukturę aplikacji frontendowej oraz definiuje poszczególne elementy modelu DOM, takie jak nagłówki, formularze czy przyciski,
- CSS (Cascading Style Sheets) odpowiada za warstwę wizualną aplikacji, w tym układ elementów, kolory, czcionki oraz ogólny wygląd interfejsu użytkownika,
- JavaScript wprowadza warstwę dynamiczną, umożliwiając manipulację modelem DOM oraz reagowanie na zdarzenia wywoływane przez użytkownika [6].

Zastosowanie języka JavaScript pozwala na dynamiczne aktualizowanie zawartości strony bez konieczności jej ponownego ładowania. Frontend może w ten sposób obsługiwać podstawowe interakcje użytkownika, takie jak walidacja danych w formularzach, wyświetlanie kalendarza czy reagowanie na kliknięcia przycisków. Bardziej złożone operacje, w szczególności te wymagające dostępu do danych lub logiki biznesowej, są przekazywane do warstwy backendowej aplikacji za pośrednictwem zapytań sieciowych [5].

Istotnym zagadnieniem w kontekście aplikacji webowych jest współbieżność (concurrency). W przypadku warstwy frontendowej każdy użytkownik posiada własną instancję aplikacji działającą w przeglądarce lub aplikacji mobilnej. Oznacza to, że frontend nie musi obsługiwać wielu użytkowników jednocześnie w ramach jednej instancji, co eliminuje problemy związane z synchronizacją zasobów oraz zarządzaniem współbieżnym dostępem do danych [6].

Bezpieczeństwo warstwy frontendowej koncentruje się na elementach widocznych dla użytkownika, takich jak formularze wejściowe, skrypty wykonywane po stronie klienta oraz przebieg procesów uwierzytelniania. Do najczęściej stosowanych mechanizmów bezpieczeństwa po stronie frontendowej należą walidacja danych wejściowych, ograniczanie możliwości wprowadzania kodu do pól tekstowych oraz stosowanie wieloskładnikowych procesów uwierzytelniania. Część odpowiedzialności za bezpieczeństwo spoczywa również na użytkowniku końcowym, który powinien dbać o poufność danych uwierzytelniających oraz bezpieczeństwo wykorzystywanego urządzenia [5].

2.4. Komunikacja frontend–backend

W nowoczesnych aplikacjach webowych komunikacja pomiędzy warstwą frontendową a backendową stanowi kluczowy element architektury systemu. Frontend odpowiada za prezentację danych oraz obsługę interakcji użytkownika, natomiast backend realizuje logikę biznesową i zapewnia dostęp do danych. Wymiana informacji pomiędzy tymi warstwami odbywa się za pośrednictwem interfejsów komunikacyjnych opartych na standardowych protokołach sieciowych [3].

Podstawowym mechanizmem komunikacji w aplikacjach webowych jest protokół HTTP, który definiuje sposób przesyłania żądań oraz odpowiedzi pomiędzy klientem a serwerem. Frontend inicjuje zapytania do backendu, korzystając z metod takich jak GET, POST, PUT czy DELETE, w zależności od rodzaju wykonywanej operacji [13]. Odpowiedzi serwera zawierają dane w ustalonym formacie, najczęściej JSON, które następnie są przetwarzane oraz prezentowane w interfejsie użytkownika.

W architekturze aplikacji typu Single Page Application komunikacja z backendem odbywa się asynchronicznie, bez konieczności ponownego ładowania całej strony. Mechanizmy takie jak Fetch API umożliwiają wysyłanie zapytań sieciowych w tle, co pozwala na dynamiczne aktualizowanie zawartości aplikacji oraz poprawę komfortu użytkowania. Taki model komunikacji sprzyja również rozdzieleniu warstwy prezentacji od logiki biznesowej [4].

Istotnym aspektem komunikacji frontend–backend jest obsługa błędów oraz zabezpieczenie przesyłanych danych. Frontend powinien reagować na kody odpowiedzi HTTP, takie jak błędy autoryzacji czy nieprawidłowe dane wejściowe, natomiast backend odpowiada za walidację danych oraz kontrolę dostępu do zasobów. W celu zapewnienia poufności i integralności danych komunikacja powinna odbywać się z wykorzystaniem protokołu HTTPS [3].

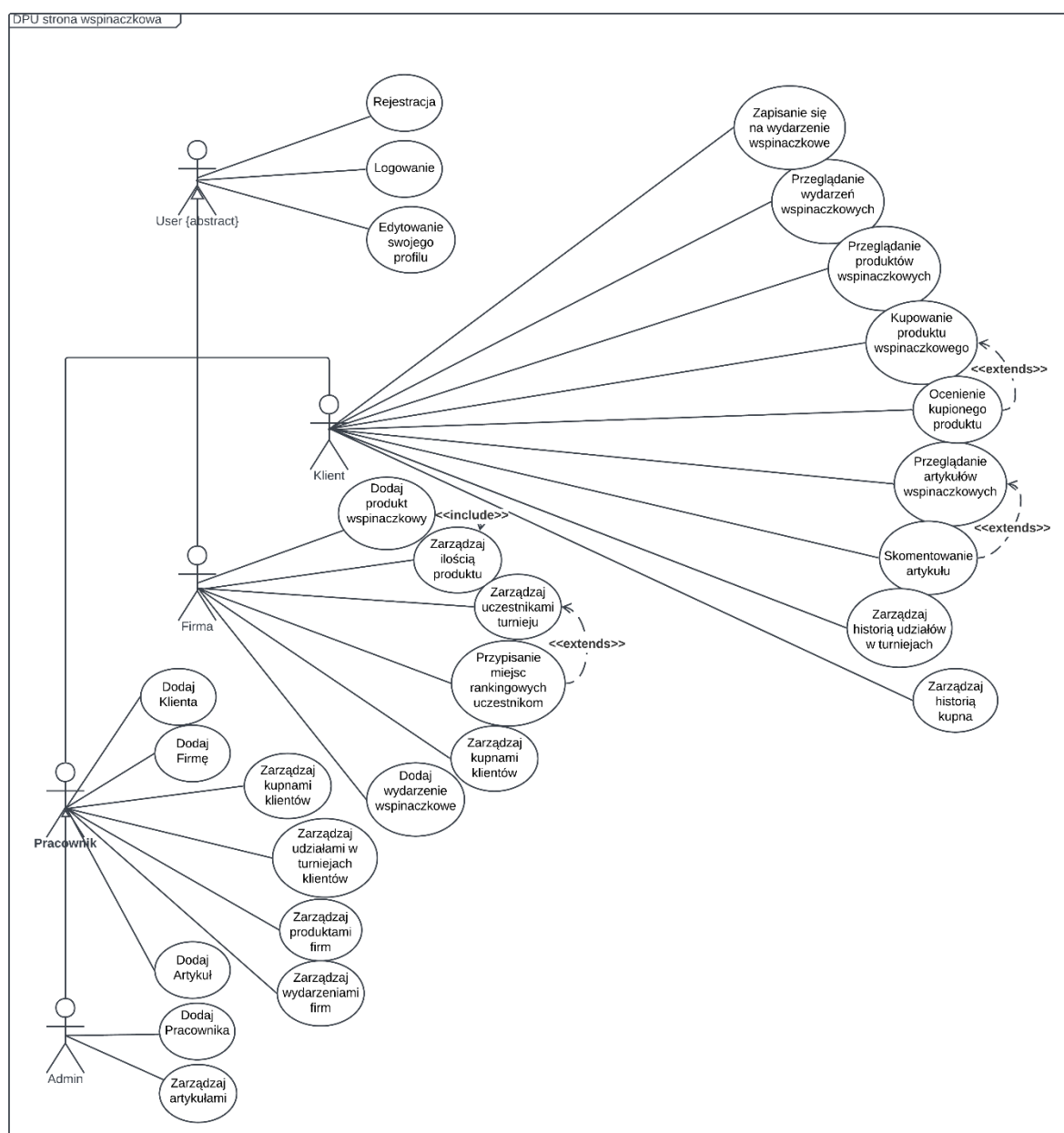
2.5. Wspinaczkowa aplikacja webowa w kontekście architektury klient–serwer

Przedstawione w niniejszym rozdziale zagadnienia związane z architekturą aplikacji webowych, modelem klient–serwer, aplikacjami typu Single Page Application oraz warstwą frontendową stanowią podstawę do zaprojektowania aplikacji webowej dedykowanej społeczności wspinaczkowej. Projektowana aplikacja wykorzystuje opisane wcześniej koncepcje w celu zapewnienia spójnego, skalowalnego oraz nowoczesnego rozwiązania informatycznego.

Aplikacja wspinaczkowa została zaprojektowana jako system działający w architekturze klient–serwer, w którym warstwa frontendowa, uruchamiana w przeglądarce internetowej użytkownika, odpowiada za prezentację treści oraz obsługę interakcji. Warstwa backendowa realizuje logikę biznesową aplikacji oraz zapewnia dostęp do danych przechowywanych w relacyjnej bazie danych. Komunikacja pomiędzy tymi warstwami odbywa się za pośrednictwem protokołu HTTP z wykorzystaniem interfejsu programistycznego API.

Ze względu na charakter aplikacji oraz konieczność dynamicznego aktualizowania treści, takich jak aktualności, wydarzenia czy oferta produktowa, przyjęto architekturę typu Single Page Application. Pozwala ona na płynne przechodzenie pomiędzy poszczególnymi widokami aplikacji bez konieczności ponownego ładowania całej strony, co przekłada się na poprawę komfortu użytkowania.

Projektowana aplikacja uwzględnia różne typy użytkowników, w tym użytkowników indywidualnych, konta firmowe oraz konta administracyjne, co wymaga odpowiedniego zarządzania dostępem do zasobów systemu. Na Rysunek 1 przedstawiono diagram przypadków użycia ukazujący funkcjonalności aplikacji dla poszczególnych aktorów:



Rysunek 1 Diagram przypadków użycia aplikacji wspinaczkowej. Źródło: opracowanie własne.

3. Wykorzystane technologie

W tym rozdziale przedstawiono technologie wykorzystane w procesie projektowania i implementacji aplikacji webowej dla społeczności wspinaczkowej. Projektowana aplikacja została zrealizowana w podejściu full-stack, co oznacza, że obejmuje zarówno warstwę frontendową, odpowiedzialną za interfejs użytkownika, jak i warstwę backendową, realizującą logikę biznesową oraz dostęp do danych.

3.1. Język JavaScript

JavaScript jest językiem programowania wykorzystywanym w projektowanej aplikacji zarówno po stronie klienta, jak i serwera. Jest to lekki, interpretowany i kompilowany w czasie rzeczywistym język programowania, który należy do najczęściej wykorzystywanych języków w tworzeniu aplikacji internetowych. Choć początkowo JavaScript był znany głównie jako język skryptowy służący do dodawania interaktywności na stronach WWW, poprzez środowisko uruchomieniowe Node.js znalazło zastosowanie również po stronie serwera, umożliwiając implementację logiki backendowej w tym samym języku [7].

JavaScript jest językiem dynamicznie typowanym, co oznacza, że zmienne nie mają przypisanego stałego typu i mogą przechowywać różne rodzaje danych w trakcie działania programu. Przykładowo, zmienna może w jednym momencie przechowywać liczbę, a w kolejnym tekst. Zastosowanie tych mechanizmów zwiększa elastyczność programowania, jednocześnie wymagając większej dbałości o poprawność typów danych [7].

Język ten posiada wbudowane mechanizmy do obsługi operacji asynchronicznych, co jest kluczowe w aplikacjach webowych, które komunikują się z serwerem lub bazą danych. Jednym z kluczowych elementów jest Promise, reprezentujący przyszły wynik operacji asynchronicznej, oraz składnia `async/await`, która ułatwia pisanie czytelnego i przejrzystego kodu asynchronicznego. W efekcie frontend może pobierać dane z backendu w tle, bez blokowania interfejsu użytkownika, a backend w Node.js obsługiwać wiele żądań jednocześnie [7].

3.2. React

React jest popularną biblioteką języka JavaScript, opracowaną przez firmę Facebook, wykorzystywaną do tworzenia nowoczesnych aplikacji internetowych oraz mobilnych. Umożliwia ona budowanie interaktywnych, stanowych i wielokrotnego użytku komponentów

interfejsu użytkownika, dzięki czemu sprawdza się w renderowaniu złożonych widoków przy zachowaniu wysokiej wydajności [8].

Fundamentalną cechą biblioteki React jest możliwość tworzenia komponentów interfejsu użytkownika przy użyciu składni JSX, która pozwala na łączenie kodu JavaScript z elementami przypominającymi HTML. JSX nie stanowi odrębnego języka programowania, lecz jest rozszerzeniem składni JavaScript, które w trakcie procesu budowania aplikacji zostaje przetłumaczone na standardowy kod JavaScript. Zastosowanie JSX znacząco upraszcza proces tworzenia komponentów. W porównaniu z pierwotnym podejściem opartym na metodzie `React.createElement`, kod szybko stawał się nieczytelny i trudny w utrzymaniu, nawet w przypadku niewielkich komponentów. Dzięki JSX kod staje się bardziej przejrzysty, łatwiejszy w zapisie, analizie oraz dalszym utrzymaniu, a także mniej podatny na błędy [9].

Model DOM (Document Object Model) stanowi wewnętrzną reprezentację dokumentu HTML w przeglądarce i ma postać struktury drzewiastej, w której każdy węzeł posiada określone właściwości i metody umożliwiające jego modyfikację za pomocą JavaScript. Choć pojedyncze operacje na DOM są stosunkowo lekkie, w nowoczesnych aplikacjach webowych częste i masowe aktualizacje struktury dokumentu mogą prowadzić do istotnych spadków wydajności, szczególnie gdy modyfikowane są elementy, które faktycznie nie uległy zmianie [9].

W celu rozwiązania tego problemu w bibliotece React wprowadzono mechanizm Virtual DOM, czyli pośrednią warstwę reprezentującą strukturę DOM w pamięci. Zmiany w stanie aplikacji powodują utworzenie nowej wersji tej reprezentacji, która następnie jest porównywana z poprzednią wersją. Na podstawie wykrytych różnic wykonywane są jedynie minimalnie niezbędne aktualizacje w rzeczywistym DOM. Ma to na celu znacząco ograniczyć liczbę kosztownych operacji manipulacji dokumentem, co przekłada się na lepszą wydajność i płynność działania aplikacji, stanowiąc jeden z kluczowych czynników popularności biblioteki React [9].

Inicjalizacja projektu React w ramach realizowanej aplikacji została wykonana przy użyciu narzędzia Create React App, które umożliwia szybkie utworzenie gotowej struktury projektu wraz z domyślną konfiguracją środowiska developerskiego (Rysunek 2).

```
npx create-react-app my-app  
cd my-app  
npm start
```

Rysunek 2 Sposób inicjalizacji i uruchomienia aplikacji React. Źródło: <https://create-react-app.dev/docs/getting-started/>

Narzędzie to automatycznie konfiguruje m.in. Webpack, Babel oraz serwer deweloperski, co pozwala skupić się na implementacji logiki aplikacji bez konieczności ręcznej konfiguracji środowiska.

3.3. CSS

Cascading Style Sheets (CSS) to język arkuszy stylów wykorzystywany do definiowania wyglądu oraz układu elementów interfejsu użytkownika w aplikacjach webowych napisanych w HTML lub XML. CSS odpowiada za takie aspekty jak rozmieszczenie elementów na stronie, kolory, czcionki, marginesy oraz responsywność interfejsu [10].

3.4. i18next

W celu zapewnienia obsługi wielu wersji językowych w aplikacji zastosowano bibliotekę i18next wraz z rozszerzeniem react-i18next, przeznaczonym do integracji z frameworkiem React. Rozwiązanie to umożliwia dynamiczną zmianę języka interfejsu użytkownika bez konieczności ponownego ładowania strony. Proces internacjonalizacji rozpoczyna się od inicjalizacji biblioteki i18next, w której definiowane są dostępne języki oraz pliki z tłumaczeniami (Rysunek 3) [11].

```
# npm  
$ npm install i18next --save  
  
# yarn  
$ yarn add i18next
```

Rysunek 3 Sposoby inicjalizacji biblioteki i18next. Źródło: <https://www.i18next.com/overview/getting-started>

Tłumaczenia przechowywane są w oddzielnych plikach konfiguracyjnych (np.JSON), pogrupowanych według języków i kluczy tekstowych. Zasoby te są przypisywane podczas inicjalizacji biblioteki i18next (Rysunek 4).

```
i18next.init({
  interpolation: {escapeValue: false},
  lng: "pl",
  resources: {
    en: {global: global_en},
    pl: {global: global_pl}
  }
})
```

Rysunek 4 Konfiguracja i18next z plikami tłumaczeń JSON. Źródło: opracowanie własne.

W komponentach aplikacji teksty interfejsu pobierane są za pomocą funkcji tłumaczącej, co pozwala na spójne i centralne zarządzanie treściami językowymi w całym systemie. Za zmianę aktywnego języka w czasie działania aplikacji odpowiada dedykowany komponent LanguageSelect, który aktualizuje bieżące ustawienia językowe aplikacji (Rysunek 5).

```
import {useTranslation} from "react-i18next";

const LanguageSelect = (props) => {
  const [translate, i18n] = useTranslation("global");

  const changeLanguage = (lang) => {
    props.setLanguage(lang)
    i18n.changeLanguage(lang);
  };

  return (
    <select value={props.language} onChange={e => changeLanguage(e.target.value)}>
      <option value="pl">Polski</option>
      <option value="en">English</option>
    </select>
  );
}

export default LanguageSelect
```

Rysunek 5 Plik LanguageSelect.js do zmiany języka. Źródło: opracowanie własne.

3.5. Node.js

Node.js to asynchroniczne środowisko uruchomieniowe JavaScript oparte na modelu zdarzeniowym, zaprojektowane z myślą o tworzeniu skalowalnych aplikacji sieciowych. W odróżnieniu od tradycyjnego modelu wielowątkowego, Node.js nie używa wątków systemowych do obsługi połączeń, co sprawia, że jest bardziej efektywny i łatwiejszy w użyciu. Większość funkcji w środowisku Node.js realizuje operacje wejścia/wyjścia w sposób asynchroniczny, co sprawia, że główny wątek wykonania nie jest blokowany, z wyjątkiem przypadków użycia synchronicznych metod dostępnych w standardowej bibliotece [12].

Node.js opiera się na pętli zdarzeń (event loop), która uruchamia się automatycznie po wykonaniu skryptu wejściowego i kończy, gdy nie ma więcej zadań do przetworzenia.

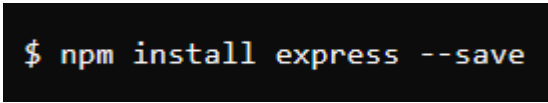
Umożliwia to tworzenie skalowalnych systemów sieciowych bez blokowania procesów oraz ryzyka występowania zakleszczeń (deadlocków), czyli sytuacji, w których dwa lub więcej procesów wzajemnie oczekują na zwolnienie zasobów i w konsekwencji żaden z nich nie może kontynuować działania [12].

Środowisko Node.js obsługuje HTTP jako podstawowy element, zoptymalizowany pod kątem strumieniowania i niskiego opóźnienia, co czyni je odpowiednim fundamentem dla bibliotek i frameworków webowych. Choć Node.js nie używa wątków, można wykorzystać wiele rdzeni procesora dzięki uruchamianiu procesów potomnych przy użyciu `child_process.fork()` lub modułu `cluster`, który umożliwia współdzielenie gniazd i równoważenie obciążenia między rdzeniami [12].

3.6. Express

Express.js to szybki, minimalistyczny i elastyczny framework webowy dla środowiska Node.js, który upraszcza tworzenie aplikacji webowych i API. Zapewnia obsługę routingu dla różnych metod HTTP (GET, POST, PUT, DELETE) oraz integrację funkcji takich jak middleware, autoryzacja czy parsowanie danych. Za pomocą modularnej architektury programista może łatwo rozszerzać aplikację i dostosować strukturę kodu do własnych potrzeb. Express jest wydajny, nie blokuje procesów serwera i dobrze współpracuje z asynchronicznym modelem Node.js, co pozwala na obsługę wielu równoczesnych połączeń [13].

Inicjalizacja projektu Express w praktyce polega na zainstalowaniu pakietu `express` i utworzeniu głównego pliku serwera, np. `index.js` (Rysunek 6).



```
$ npm install express --save
```

Rysunek 6 Komenda do instalacji frameworku express. Źródło: <https://expressjs.com/>

W pliku `index.js` definiuje się podstawowy serwer i trasę testową (Rysunek 7).

```
import express from 'express'

const app = express()

app.get('/', (req, res) => {
  res.send('Hello World')
})

app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000')
})
```

Rysunek 7 Podstawowy plik `index.js` inicjalizujący serwer Express. Źródło: <https://www.npmjs.com/package/express>

Po uruchomieniu serwera za pomocą komendy `node index.js` aplikacja jest gotowa do przyjmowania żądań http.

3.7. Relacyjna baza danych MySQL

MySQL jest popularnym systemem zarządzania relacyjnymi bazami danych (RDBMS), rozwijanym jako oprogramowanie o otwartym kodzie źródłowym, który umożliwia przechowywanie danych w postaci tabel powiązanych ze sobą relacjami. Dane w MySQL są organizowane w wiersze i kolumny, co ułatwia ich wyszukiwanie, filtrowanie i sortowanie przy użyciu języka SQL (Structured Query Language). System obsługuje mechanizmy transakcji, klucze główne i obce, indeksy oraz zapewnia funkcje kontroli integralności danych i bezpieczeństwa. Pozwala to aplikacjom webowym działać wydajnie i niezawodnie, nawet przy dużych wolumenach danych [14].

MySQL wspiera różnorodne typy danych, od prostego tekstu i liczb, przez daty, aż po bardziej złożone typy, takie jak JSON. Obsługuje transakcje ACID (atomicity, consistency, isolation, durability), co gwarantuje spójność i niezawodność operacji nawet w przypadku awarii systemu. Popularność MySQL wynika z wysokiej wydajności, prostoty obsługi, dobrej skalowalności oraz szerokiego wsparcia społeczności deweloperskiej, co sprawia, że system ten znajduje zastosowanie zarówno w niewielkich projektach, jak i w rozbudowanych systemach internetowych [15].

3.8. Sequelize

Sequelize to biblioteka typu Object-Relational Mapping (ORM) przeznaczona dla środowiska Node.js, która umożliwia pracę z relacyjnymi bazami danych przy użyciu

obiekтового modelu programistycznego. Obsługuje popularne systemy baz danych, takie jak MySQL, PostgreSQL, MariaDB, SQLite oraz Microsoft SQL Server [16].

Głównym zadaniem Sequelize jest ukrycie bezpośredniego użycia języka SQL poprzez mapowanie tabel bazodanowych na obiekty JavaScript, nazywane modelami. Każdy model reprezentuje pojedynczą tabelę w bazie danych, natomiast jego właściwości odpowiadają kolumnom tej tabeli [16].

Aby rozpocząć pracę z biblioteką Sequelize, należy zainstalować ją wraz z odpowiednim sterownikiem bazy danych (Rysunek 8).

```
npm install sequelize sqlite3  
# or  
yarn add sequelize sqlite3
```

Rysunek 8 Instalacja zależności Sequelize. Źródło: <https://sequelize.org/>

Modele danych w Sequelize przechowywane są w katalogu models. Każdy model reprezentuje jedną tabelę w bazie danych i zawiera definicję jej pól, typów danych oraz relacji z innymi modelami.

Przykładem jest Model User w Sequelize, która pełni rolę reprezentacji tabeli Users w bazie danych i odpowiada za przechowywanie podstawowych informacji o użytkownikach systemu. Każdy użytkownik posiada określone właściwości, takie jak imię, adres e-mail oraz hasło, które nie mogą być puste (Rysunek 9).

```
const User= sequelize.define("User", {
  name: {
    type: DataTypes.STRING,
    allowNull: false,
    validate: {
      notEmpty: true,
    },
  },
  email: {
    type: DataTypes.STRING,
    allowNull: false,
    validate: {
      notEmpty: true,
    },
  },
  password: {
    type: DataTypes.STRING,
    allowNull: false,
    validate: {
      notEmpty: true,
    },
  },
});
```

Rysunek 9 Model User z zdefiniowanymi parametrami. Źródło: opracowanie własne.

W modelu zastosowano także tzw. hooki, czyli funkcje wywoływane w określonych momentach cyklu życia obiektu. W przypadku encji User przed zapisaniem nowego rekordu do bazy danych oraz przed każdą aktualizacją hasła wykonywana jest operacja jego kryptograficznego skrótu (ang. hashing). Proces ten polega na jednokierunkowym przekształceniu hasła do postaci nieodwracalnej, co zwiększa bezpieczeństwo przechowywanych danych uwierzytelniających. Znacząco zwiększa to bezpieczeństwo danych, jednocześnie upraszczając kod aplikacji, gdyż nie wymaga ręcznego wywoływania tych operacji w logice biznesowej (Rysunek 10).

```

hooks: {
  async beforeCreate(user) {
    if (user.password) {
      user.password = await bcrypt.hash(user.password, 10);
    }
  },
  async beforeUpdate(user) {
    if (user.changed("password")) {
      user.password = await bcrypt.hash(user.password, 10);
    }
  },
}
}

```

Rysunek 10 Zdefiniowane hooki do hashowania hasła. Źródło: opracowanie własne.

Model definiuje również metodę instancji checkPassword, umożliwiającą weryfikację wprowadzonego hasła względem wartości zapisanej w bazie. Funkcja ta jest wykorzystywana podczas procesu logowania użytkownika i pozwala na bezpieczne uwierzytelnianie (Rysunek 11).

```

User.prototype.checkPassword = async function(password) {
  return await bcrypt.compare(password, this.password);
}

```

Rysunek 11 Funkcja checkPassword służąca do weryfikacji poprawności podanego hasła. Źródło: opracowanie własne.

Ponadto, model User posiada relacje z innymi modelami w systemie – klientem, firmą i deweloperem. Każdy użytkownik może być przypisany do jednej z tych ról, a usunięcie użytkownika powoduje automatyczne usunięcie powiązanych danych w powiązanych tabelach, co zapewnia spójność bazy danych (Rysunek 12).

```

User.associate = (models) => {
  User.hasOne(models.Client, {
    foreignKey: "userID",
    onDelete: "CASCADE",
  });
  User.hasOne(models.Brand, {
    foreignKey: "userID",
    onDelete: "CASCADE",
  });
  User.hasOne(models.Developer, {
    foreignKey: "userID",
    onDelete: "CASCADE",
  });
};

```

Rysunek 12 Asocjacje modelu User z innymi encjami w systemie. Źródło: opracowanie własne.

Plik `index.js` pełni w projekcie rolę centralnego modułu inicjalizującego połączenie z bazą danych i przygotowującego modele do dalszego użycia w aplikacji. Na podstawie środowiska uruchomieniowego tworzona jest instancja Sequelize, która odpowiada za połączenie z odpowiednią bazą danych.

Kolejnym krokiem jest automatyczne wczytanie wszystkich modeli z katalogu `models`. Każdy plik modelu jest importowany, a następnie dodawany do obiektu `db`, który służy jako centralny punkt dostępu do wszystkich modeli w projekcie. W tym miejscu sprawdzane są także relacje między modelami i wywoływana jest metoda `associate`, jeśli została zdefiniowana. Pozwala to na automatyczne tworzenie powiązań między tabelami w bazie danych (Rysunek 13).

```

fs
.readdirSync(__dirname)
  .filter(file => {
    return (
      file.indexOf('.') !== 0 &&
      file !== basename &&
      file.slice(-3) === '.js' &&
      file.indexOf('.test.js') === -1
    );
  })
  .forEach(file => {
    const model = require(path.join(__dirname, file))(sequelize, Sequelize.DataTypes);
    db[model.name] = model;
  });

Object.keys(db).forEach(modelName => {
  if (db[modelName].associate) {
    db[modelName].associate(db);
  }
});

```

Rysunek 13 Plik `index.js` odpowiedzialny za ładowanie modeli i tworzenie powiązań między nimi. Źródło: opracowanie własne.

Po uruchomieniu aplikacji następuje synchronizacja modeli z bazą danych za pomocą metody pokazanej na Rysunek 14.

```

db.sequelize.sync().then((req) => {
  server.listen(3001, () => {
    console.log("listening on port 3001");
  });
})

```

Rysunek 14 Wywołanie `db.sequelize.sync()` inicjalizujące tabele w bazie danych na podstawie zdefiniowanych modeli. Źródło: opracowanie własne.

Metoda `sync()` sprawdza, czy odpowiadające modelom tabele istnieją w bazie danych, a w przypadku ich braku tworzy je automatycznie. Pozwala to na szybkie uruchomienie aplikacji bez ręcznego tworzenia struktury bazy danych, co jest szczególnie przydatne na etapie rozwoju i testowania systemu.

3.9. Cypress

Cypress to nowoczesne narzędzie do automatyzacji testów aplikacji webowych, umożliwiające testowanie interfejsu użytkownika w czasie rzeczywistym. Umożliwia wykonywanie testów end-to-end (E2E), integracyjnych oraz jednostkowych, zapewniając pełną kontrolę nad przeglądarką i środowiskiem aplikacji [17].

W odróżnieniu od tradycyjnych narzędzi testowych, Cypress działa bezpośrednio w środowisku przeglądarki, co umożliwia precyzyjne monitorowanie interakcji użytkownika z aplikacją oraz obserwację rzeczywistego przebiegu testów end-to-end. Narzędzie pozwala na

łatwe symulowanie kliknięć, wpisywania tekstu, przewijania strony czy weryfikowania elementów DOM [17].

Aby rozpocząć korzystanie z Cypress w projekcie, należy najpierw zainstalować pakiet przy użyciu npm (Rysunek 15).

```
npm install cypress --save-dev
```

Rysunek 15 Komenda do instalacji narzędzia Cypress. Źródło:

<https://docs.cypress.io/app/get-started/install-cypress>.

Po instalacji można otworzyć interfejs graficzny Cypress, który umożliwia przeglądanie dostępnych testów i uruchamianie ich w wybranej przeglądarce (Rysunek 16).

```
npx cypress open
```

Rysunek 16 Komenda do uruchomienia graficznego interfejsu Cypress. Źródło:

<https://docs.cypress.io/app/get-started/install-cypress>.

Po uruchomieniu, Cypress wykonuje testy zapisane w katalogu `cypress/e2e/`, generując raporty.

3.10. JestJS

Jest to popularny framework testowy dla języka JavaScript, wykorzystywany do testowania aplikacji oraz bibliotek napisanych w tym języku, w tym również aplikacji opartych na React. Narzędzie oferuje rozbudowany zestaw matcherów (asercji), które umożliwiają precyzyjne sprawdzanie oczekiwanych rezultatów działania kodu w różnych scenariuszach [18].

Jest umożliwia organizowanie testów w tzw. zestawy testowe (test suites), co pozwala na logiczne grupowanie przypadków testowych i sprzyja utrzymaniu czytelnej struktury kodu testów. Omawiany framework oferuje funkcjonalności takie jak asercje, mockowanie modułów, śledzenie wywołań funkcji oraz możliwość izolowania testów [18].

Aby rozpocząć pracę z frameworkiem Jest, należy zainstalować go jako zależność deweloperską projektu przy użyciu menedżera pakietów npm (Rysunek 17).

```
npm install --save-dev jest
```

Rysunek 17 Komenda do instalacji frameworku Jest. Źródło: <https://jestjs.io/docs/getting-started>

Po zakończeniu instalacji, w pliku `package.json` definiuje się skrypt umożliwiający uruchamianie testów (Rysunek 18).

```
{  
  "scripts": {  
    "test": "jest"  
  }  
}
```

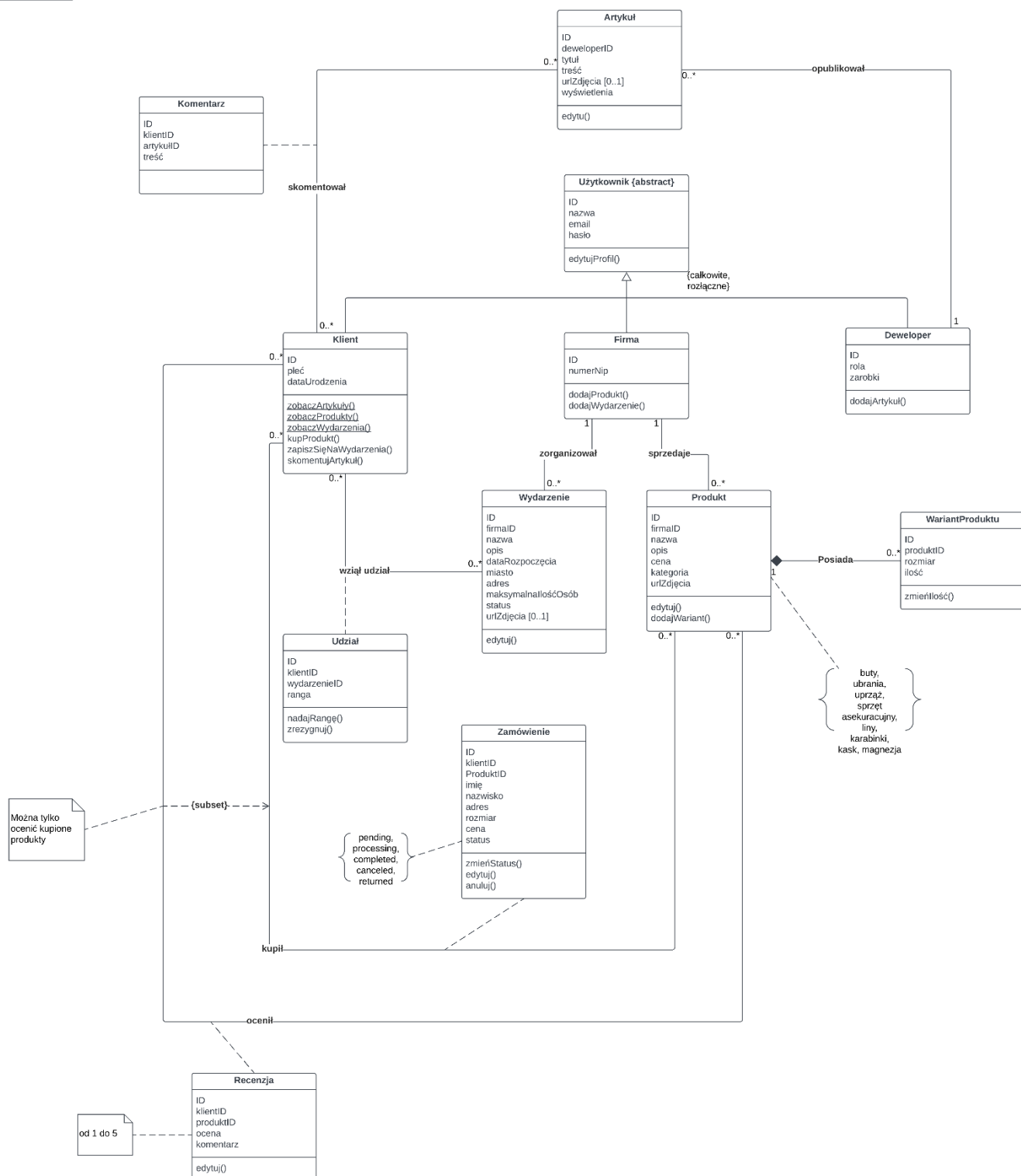
Rysunek 18 Definicja skryptu umożliwiająca uruchamianie testów w pliku `package.json`. Źródło: <https://jestjs.io/docs/getting-started>

Testy mogą być umieszczane w plikach z rozszerzeniem `„.test.js”`. Uruchomienie testów odbywa się za pomocą polecenia `„npm test”`.

4. Implementacja

4.1. Diagram klas

Na początku rozdziału przedstawiono na Rysunek 19 diagram klas ilustrujący strukturę implementacyjną systemu oraz relacje pomiędzy jego głównymi elementami. Diagram ten stanowi podstawę do dalszego opisu poszczególnych modułów aplikacji oraz ich funkcjonowania.



Rysunek 19 Diagram klas systemu. Źródło: opracowanie własne.

4.2. Mechanizm uwierzytelniania i autoryzacji użytkowników

W projektowanej aplikacji zastosowano oddzielne mechanizmy logowania dla poszczególnych typów użytkowników, takich jak klient, firma oraz admin/deweloper (Rysunek 20). Taki podział wynika przede wszystkim z potrzeby zwiększenia bezpieczeństwa systemu oraz optymalizacji procesu uwierzytelniania.



The image shows a login interface with a dark green background. At the top, the word "Logowanie" is written in white. Below it are two light gray input fields with green placeholder text: "e-mail" and "hasło". A gray button with the text "zaloguj się" is positioned below the fields. At the bottom of the screen, there are three white text links: "Rejestruj", "platforma dla firm", and "Portal pracownika".

Rysunek 20 Ekran logowania klienta z odnośnikami do logowania dla innych ról. Źródło: opracowanie własne.

Każda z ról użytkowników posiada odmienny zakres uprawnień oraz dostęp do różnych funkcjonalności aplikacji. Rozdzielenie logowania pozwala na jednoznaczne określenie roli użytkownika już na etapie uwierzytelniania, co upraszcza dalszą obsługę autoryzacji oraz ogranicza ryzyko nieautoryzowanego dostępu do zasobów systemu.

Dodatkowo takie podejście umożliwia lepszą separację logiki biznesowej po stronie backendu, a także uproszczenie warstwy frontendowej poprzez kierowanie użytkowników do dedykowanych interfejsów dostosowanych do ich roli. W efekcie system staje się bardziej czytelny, bezpieczny i łatwiejszy w dalszym rozwoju.

4.3. Mechanizm kontroli dostępu w warstwie frontendowej

W aplikacji frontendowej zaimplementowano mechanizm kontroli dostępu oparty na tokenach JWT. Mechanizm ten został umieszczony w komponencie Layout, który pełni rolę wspólnego kontenera dla chronionych widoków aplikacji. Poniżej zamieszczono fragment komponentu Layout odpowiedzialny za weryfikację tokenu JWT (Rysunek 21).

```
useEffect(() => {  
  const token = localStorage.getItem("token");  
  // W przypadku braku tokena użytkownik zostaje przekierowany na stronę główną.  
  if (!token) {  
    navigate("/");  
    return;  
  }  
  // proces autentykacji  
  fetch("http://localhost:3001/profileClient", {  
    headers: { Authorization: "Bearer " + token },  
  })  
    .then(res => res.json())  
    .then(data => {  
      props.setUser(data);  
    })  
    // przy nieudanej próbie autentykacji usuwane jest token i odsyła  
    // użytkownika do strony głównej  
    .catch(() => { localStorage.removeItem("token");  
      navigate("/"); });  
}, []);
```

Rysunek 21 Weryfikacja tokenu JWT. Źródło: opracowanie własne.

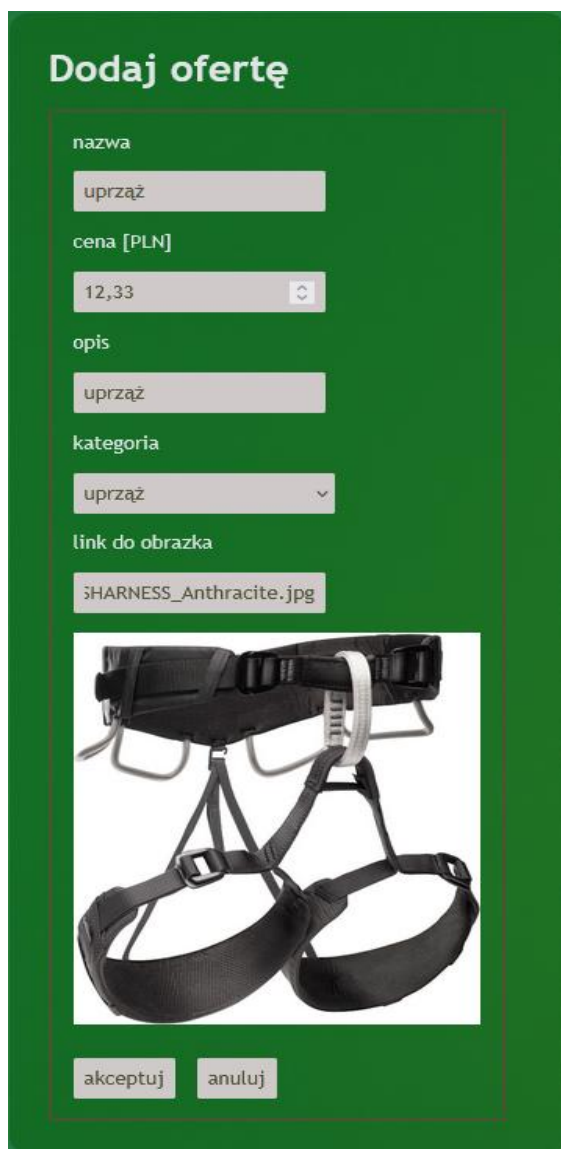
Podczas inicjalizacji komponentu sprawdzana jest obecność tokenu JWT w pamięci lokalnej przeglądarki. W przypadku jego braku użytkownik zostaje automatycznie przekierowany do strony głównej aplikacji. Jeśli token istnieje, wysyłane jest żądanie do endpointu backendowego w celu weryfikacji jego poprawności oraz pobrania danych użytkownika.

W sytuacji, gdy token jest nieprawidłowy, wygasły lub wystąpi błąd komunikacji z serwerem, token zostaje usunięty z pamięci lokalnej, a użytkownik zostaje wylogowany i przekierowany poza chronioną część aplikacji. Takie rozwiązanie zapewnia podstawowy poziom bezpieczeństwa oraz uniemożliwia dostęp do funkcjonalności aplikacji osobom nieautoryzowanym.

4.4. Moduł zakupów i sprzedaży

Moduł zakupów i sprzedaży umożliwia pełną interakcję pomiędzy klientami a firmami w kontekście zarządzania produktami oraz zamówieniami. Proces rozpoczyna się od dodania

produktu przez firmę (Rysunek 22). W tym momencie definiowane są jego podstawowe właściwości, a następnie można wprowadzić warianty, takie jak rozmiar czy ilość. W efekcie oferta jest kompletna i może zostać zaprezentowana użytkownikom końcowym.



Dodaj ofertę

nazwa
uprząż

cena [PLN]
12,33

opis
uprząż

kategoria
uprząż

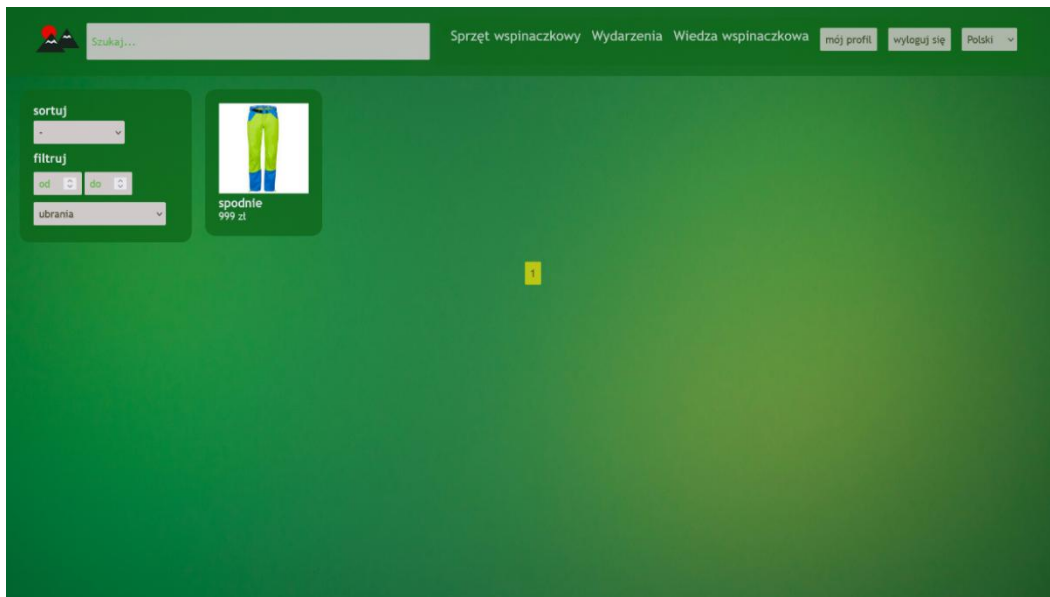
link do obrazka
iHARNESS_Anthrachte.jpg



akceptuj anuluj

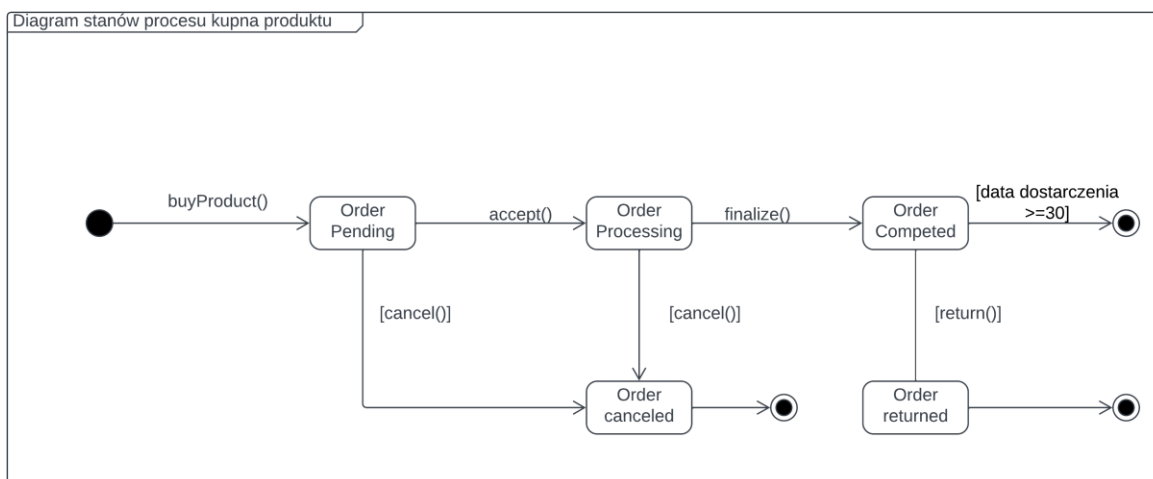
Rysunek 22 Ekran dodania produktu firmy. Źródło: opracowanie własne.

Klienci mogą następnie przeglądać dostępne produkty (Rysunek 23) i dokonywać zakupów. Po złożeniu zamówienia jego status zostaje oznaczony jako pending, co informuje, że oczekuje ono na dalszą realizację. Firma, po zapoznaniu się z zamówieniem, rozpoczyna jego realizację, zmieniając status na processing, co odzwierciedla etap aktywnej obsługi zamówienia.



Rysunek 23 Widok katalogu produktów z perspektywy klienta. Źródło: opracowanie własne.

W systemie przewidziano również możliwość anulowania zamówienia. Klient może to zrobić, gdy zamówienie jest w statusie pending, natomiast firma ma taką możliwość, gdy zamówienie jest w statusie processing. Po zakończeniu realizacji firma zmienia status zamówienia na completed, co pozwala klientowi ocenić zakupiony produkt. Dodatkowo klient ma 30 dni na zgłoszenie zwrotu, wówczas status zamówienia zostaje zmieniony na returned. Szczegółowy przebieg procesu zakupowego wraz ze zmianami stanów zamówienia zaprezentowano na Rysunek 24.



Rysunek 24 Diagram stanów procesu zamówienia. Źródło: opracowanie własne.

4.5. Moduł wydarzeń i turniejów

Proces obsługi wydarzeń w systemie rozpoczyna się od firmy, która ma możliwość utworzenia nowego wydarzenia. Podczas dodawania wydarzenia określone są jego podstawowe parametry, takie jak nazwa, opis, data rozpoczęcia, limit uczestników oraz status początkowy ustawiany jako upcoming. Na tym etapie wydarzenie jest widoczne dla klientów, jednak jego realizacja jeszcze się nie rozpoczęła (Rysunek 25).

dodaj wydarzenie

nazwa
WLB Crux

opis
zapraszamy na turniej

data
12. 12. 2026, 14:00

miasto
Warszawa

adres
Hoża 51

maxCapacity
30

status
nadchodzące

link do obrazka
-1-1-1-1-1-1024x534.jpeg



akceptuj anuluj

Rysunek 25 Ekran dodania wydarzenia firmy. Źródło: opracowanie własne.

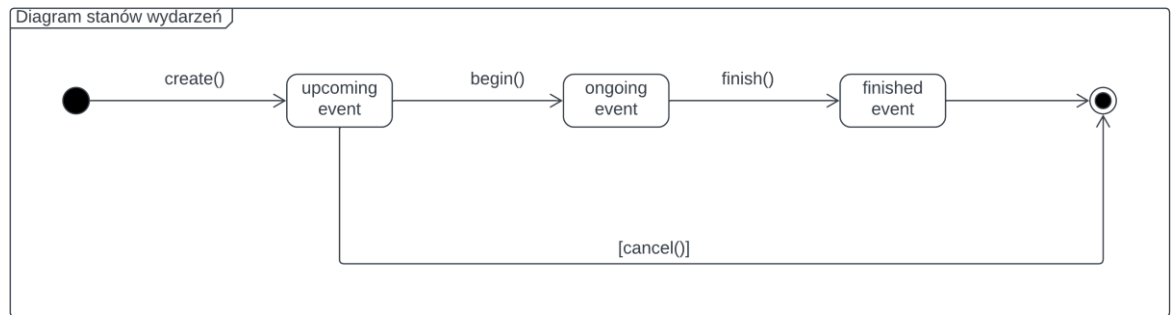
Klient może zapisać się na wydarzenie, o ile jego status pozostaje upcoming oraz nie został przekroczony limit dostępnych miejsc. System weryfikuje liczbę aktualnych uczestników oraz uniemożliwia zapis w przypadku braku wolnych miejsc. Po poprawnym zapisie klient zostaje przypisany do listy uczestników wydarzenia (Rysunek 26).



Rysunek 26 Ekran zapisu na wydarzenie. Źródło: opracowanie własne.

W momencie rozpoczęcia wydarzenia firma zmienia jego status na onGoing. W trakcie trwania wydarzenia klienci nadal mogą zapisywać się, pod warunkiem że dostępne są wolne miejsca. Dodatkowo firma uzyskuje możliwość przypisywania uczestnikom miejsc rankingowych, co pozwala na częściowe lub bieżące uzupełnianie wyników jeszcze przed zakończeniem wydarzenia.

Po zakończeniu wydarzenia jego status zostaje zmieniony na finished. Od tego momentu zapisy klientów nie są już możliwe, a firma finalizuje proces poprzez uzupełnienie rankingu wszystkich uczestników. Ranking stanowi końcowy rezultat wydarzenia i jest dostępny do wglądu dla klientów. Cały proces zmiany stanów wydarzenia został przedstawiony na Rysunek 27.



Rysunek 27 Diagram stanów wydarzeń. Źródło: opracowanie własne

4.6. Moduł artykułów i aktualności

Moduł aktualności odpowiada za publikowanie treści informacyjnych oraz budowanie interakcji pomiędzy deweloperami a klientami platformy. Proces rozpoczyna się od dodania artykułu przez dewelopera, który uzupełnia jego tytuł, treść oraz opcjonalne materiały graficzne (Rysunek 28). Po zapisaniu artykuł staje się publicznie dostępny i widoczny dla wszystkich użytkowników.

Dodaj article

tytuł

kontent

obrazek



Rysunek 28 Ekran dodania artykułu. Źródło: opracowanie własne.

Klienci mają możliwość zapoznania się z opublikowanymi artykułami oraz dodawania komentarzy pod wybranymi wpisami. Komentarze są bezpośrednio powiązane z konkretnym artykułem, co pozwala na prowadzenie dyskusji oraz wyrażanie opinii na temat publikowanych treści (Rysunek 29).



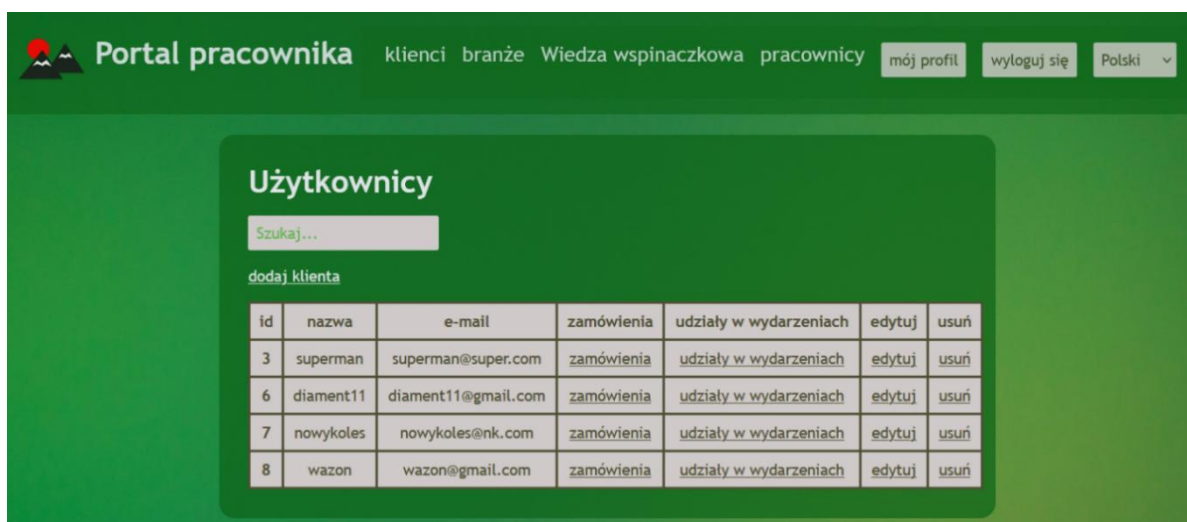
Rysunek 29 Widok artykułu wraz z komentarzami. Źródło: opracowanie własne.

4.7. Panel administracyjny i zarządzanie systemem

Panel administracyjny stanowi centralny element zarządzania systemem i przeznaczony jest dla użytkownika o najwyższych uprawnieniach – administratora. Jego zadaniem jest umożliwienie pełnej kontroli nad danymi aplikacji oraz nad kontami pozostałych użytkowników, co jest istotne z punktu widzenia bezpieczeństwa, utrzymania porządku w systemie oraz reagowania na nieprawidłowości.

Administrator posiada rozszerzone uprawnienia, które pozwalają na zarządzanie wszystkimi typami użytkowników występującymi w systemie, w tym klientami, firmami oraz deweloperami. W ramach panelu administracyjnego możliwe jest przeglądanie list użytkowników, edycja ich danych, blokowanie lub usuwanie kont, a także kontrola nad podmiotami takimi jak firmy oferujące produkty lub organizujące wydarzenia. Pozwala to administratorowi na skuteczne reagowanie na nadużycia, duplikację kont oraz naruszenia regulaminu platformy.

Panel administracyjny umożliwia również nadzór nad kluczowymi elementami systemu, takimi jak produkty, wydarzenia czy artykuły (Rysunek 30). Administrator może usuwać treści niezgodne z zasadami platformy, korygować błędne dane oraz podejmować działania porządkowe w sytuacjach wyjątkowych.



Rysunek 30 Panel administracyjny do zarządzania klientami. Źródło: opracowanie własne.

Niezależnie od panelu administracyjnego, każdy zalogowany użytkownik systemu posiada dostęp do własnego profilu, w którym może zarządzać podstawowymi informacjami dotyczącymi swojego konta. Funkcjonalności te obejmują między innymi możliwość zmiany hasła, adresu e-mail oraz innych danych osobowych. Rozdzielenie zarządzania własnym kontem od uprawnień administracyjnych pozwala zachować przejrzystość systemu oraz jasno określić zakres odpowiedzialności poszczególnych ról użytkowników.

5. Testowanie aplikacji

Jednym z kluczowych etapów procesu wytwarzania oprogramowania jest jego testowanie, którego celem jest weryfikacja poprawności działania systemu oraz wykrywanie potencjalnych błędów na możliwie wczesnym etapie rozwoju. Testowanie pozwala upewnić się, że aplikacja spełnia założone wymagania funkcjonalne i нефункционалне, а także że nowe zmiany nie zakłócają poprawnego działania wcześniej zaimplementowanych mechanizmów.

W celu zapewnienia izolacji środowiska testowego od środowiska produkcyjnego, aplikacja backendowa uruchamiana w trybie testowym korzysta z oddzielnej konfiguracji połączenia z bazą danych (Rysunek 31).

```
{
  "development": {
    "username": "root",
    "password": "██████████",
    "database": "inz",
    "host": "127.0.0.1",
    "dialect": "mysql"
  },
  "test": {
    "username": "root",
    "password": "██████████",
    "database": "inz_test",
    "host": "localhost",
    "dialect": "mysql"
  },
  "production": {
    "username": "root",
    "password": null,
    "database": "database_production",
    "host": "127.0.0.1",
    "dialect": "mysql"
  }
}
```

Rysunek 31 Konfiguracja połączenia z testową bazą danych. Źródło: opracowanie własne.

Aby uruchomić aplikację w trybie testowym zastosowano oddzielny plik startowy backendu, przeznaczony wyłącznie do wykonywania testów. Plik ten zawiera konfigurację środowiska testowego, w tym ustawienie zmiennej środowiskowej określającej tryb pracy aplikacji (Rysunek 32). Pozwala to na jednoznaczne rozróżnienie pomiędzy uruchomieniem systemu w środowisku deweloperskim a środowiskiem testowym.

```
process.env.NODE_ENV = "test"
```

Rysunek 32 Fragment kodu odpowiedzialny za przełączenie środowiska na testowe. Źródło: opracowanie własne.

W ramach testów backendowych oraz testów end-to-end przygotowano zestaw scenariuszy testowych, które weryfikują kluczowe funkcjonalności aplikacji. Każdy test został przeanalizowany pod kątem oczekiwanego zachowania systemu. W celu zwiększenia czytelności opisów, scenariusze testowe zestawiono w formie tabelarycznej, zawierającej nazwę testu, fragment kodu oraz opis oczekiwanego rezultatu.

Szczegółowe zestawienia scenariuszy przedstawiono w tabelach Tabela 1–Tabela 15, natomiast rezultaty ich wykonania zaprezentowano na rysunkach Rysunek 33–Rysunek 47. Takie zestawienie umożliwia bezpośrednie porównanie założeń testowych z uzyskanymi rezultatami oraz ułatwia analizę poprawności działania poszczególnych funkcjonalności systemu.

5.1. Testy API i testy integracyjne backendu za pomocą JestJS

Tabela 1 Scenariusze testowe rejestrowania - backend

Nazwa testu	Kod	Opis
create new client	<pre>const newClient = { name: "mockUser", email: "mockUser@mock.com", password: "mockUser", birthDate: "2002-11-13", gender: "M", } const res = await request(routes) .put('/putClient') .send(newClient) .set("Accept", "application/json")</pre>	Po poprawnym utworzeniu klienta oczekiwanym rezultatem jest odpowiedź serwera z kodem statusu 200.

	<code>expect(res.statusCode).toBe(200)</code>	
create new brand	<pre>const newBrand = { name: "mockBrand", email: "mockBrand@mock.com", password: "mockBrand", nipNumber: 212031405, } const res = await request(routes) .put('/putBrand') .send(newBrand) .set("Accept", "application/json") expect(res.statusCode).toBe(200)</pre>	Po poprawnym utworzeniu firmy oczekiwanym rezultatem jest odpowiedź serwera z kodem statusu 200.
create new developer	<pre>const newDeveloper = { name: "mockDeveloper", email: "mockDeveloper@mock.com", password: "mockDeveloper", role: 'junior', salary: 7000 } const res = await request(routes) .put('/putDeveloper') .send(newDeveloper) .set("Accept", "application/json") expect(res.statusCode).toBe(200)</pre>	Po poprawnym utworzeniu dewelopera oczekiwanym rezultatem jest odpowiedź serwera z kodem statusu 200.
400 name too short	<pre>const newClient = { name: "c", email: "mockUser1@mock.com", password: "mockUser", birthDate: "2002-11-13", gender: "M", } const res = await request(routes) .put('/putClient')</pre>	W przypadku niepoprawnej walidacji nazwy, oczekiwanym rezultatem jest odpowiedź serwera z kodem statusu 400 oraz komunikatem

	<pre> .send(newClient) .set("Accept","application/json") expect(res.statusCode).toBe(400) expect(res.body.error).toBe('Name must be at least 3 characters long') </pre>	błędu „Name must be at least 3 characters long”.
400 invalid email format	<pre> const newBrand = { name: "mockBrand1", email: "mockBrandcom", password: "mockBrand", nipNumber: 212031405, } const res = await request(routes) .put('/putBrand') .send(newBrand) .set("Accept","application/json") expect(res.statusCode).toBe(400) expect(res.body.error).toBe('Invalid email format') </pre>	W przypadku niepoprawnej walidacji maila, oczekiwanym rezultatem jest odpowiedź serwera z kodem statusu 400 oraz komunikatem błędu „Invalid email format”.
400 password too short	<pre> const newDeveloper = { name: "mockDeveloper1", email: "mockDeveloper1@mock.com", password: "mock", role: 'junior', salary: 7000 } const res = await request(routes) .put('/putDeveloper') .send(newDeveloper) .set("Accept","application/json") expect(res.statusCode).toBe(400) expect(res.body.error).toBe('Password must be at least 8 characters long') </pre>	W przypadku niepoprawnej walidacji hasła, oczekiwanym rezultatem jest odpowiedź serwera z kodem statusu 400 oraz komunikatem błędu „Password must be at least 8 characters long”.

400 birthday from the future	<pre>const newClient = { name: "mockUser1", email: "mockUser1@mock.com", password: "mockUser", birthDate: "2052-11-13", gender: "M", } const res = await request(routes) .put('/putClient') .send(newClient) .set("Accept", "application/json") expect(res.statusCode).toBe(400) expect(res.body.error).toBe('Birth date cannot be in the future')</pre>	W przypadku niepoprawnej walidacji daty urodzenia, oczekiwanym rezultatem jest odpowiedź serwera z kodem statusu 400 oraz komunikatem błędu „Birth date cannot be in the future”.
400 same email	<pre>const newClient = { name: "mockUser1", email: "mockUser@mock.com", password: "mockUser", birthDate: "2022-11-13", gender: "M", } const res = await request(routes) .put('/putClient') .send(newClient) .set("Accept", "application/json") expect(res.statusCode).toBe(400) expect(res.body.error).toBe('Email already exists')</pre>	W przypadku próby utworzenia konta z adresem e-mail, który już istnieje w systemie, oczekiwanym rezultatem jest odpowiedź serwera z kodem statusu 400 oraz komunikatem błędu „Email already exists”.
400 same name	<pre>const newClient = { name: "mockUser", email: "mockUser1@mock.com", password: "mockUser", birthDate: "2022-11-13",</pre>	W przypadku próby utworzenia konta nazwą, która już istnieje w systemie,

	<pre> gender: "M", } const res = await request(routes) .put('/putClient') .send(newClient) .set("Accept", "application/json") expect(res.statusCode).toBe(400) expect(res.body.error).toBe('Name already exists') </pre>	oczekiwanym rezultatem jest odpowiedź serwera z kodem statusu 400 oraz komunikatem błędu „Name already exists”.
--	--	--

Źródło: opracowanie własne.

```

Register
Sucess cases
✓ create new client (85 ms)
✓ create new brand (99 ms)
✓ create new developer (92 ms)
Errors
✓ 400 name too short (8 ms)
✓ 400 invalid email format (8 ms)
✓ 400 password too short (7 ms)
✓ 400 birthday from the future (5 ms)
✓ 400 same email (8 ms)
✓ 400 same name (11 ms)
Login

```

Rysunek 33 Wyniki testów scenariusza rejestrowania. Źródło: opracowanie własne.

Tabela 2 Scenariusze testowe logowanie - backend

Nazwa testu	Kod	Opis
client login	<pre> const client= { email: "mockUser@mock.com", password: "mockUser" } const res = await request(routes) .post('/client') .send(client) </pre>	Po pomyślnym zalogowaniu klienta oczekuje się odpowiedzi serwera z kodem statusu 200 oraz zwróceniem

	<pre> .set("Accept","application/json") expect(res.statusCode).toBe(200) expect(res.body.user).toBeDefined() </pre>	danych użytkownika.
brand login	<pre> const brand= { email: "mockBrand@mock.com", password: "mockBrand" } const res = await request(routes) .post('/brand') .send(brand) .set("Accept","application/json") expect(res.statusCode).toBe(200) expect(res.body.brand).toBeDefined() </pre>	Po pomyślnym zalogowaniu firmy oczekuje się odpowiedzi serwera z kodem statusu 200 oraz zwróceniem danych firmy.
developer login	<pre> const developer= { email: "mockDeveloper@mock.com", password: "mockDeveloper" } const res = await request(routes) .post('/developer') .send(developer) .set("Accept","application/json") expect(res.statusCode).toBe(200) expect(res.body.dev).toBeDefined() </pre>	Po pomyślnym zalogowaniu dewelopera oczekuje się odpowiedzi serwera z kodem statusu 200 oraz zwróceniem danych dewelopera.
400 user not found	<pre> const client= { email: "wrongEmail@mock.com", password: "mockUser" } const res = await request(routes) .post('/client') .send(client) .set("Accept","application/json") expect(res.statusCode).toBe(400) </pre>	Przy niepoprawnym emailu oczekuje się odpowiedzi serwera z kodem statusu 400 oraz komunikatem błędu „client not found”

	expect(res.body.err).toBe('client not found')	
400 wrong password	<pre>const client= { email: "mockUser@mock.com", password: "wrongPassword" } const res = await request(routes) .post('/client') .send(client) .set("Accept","application/json") expect(res.statusCode).toBe(400) expect(res.body.err).toBe('wrong password')</pre>	Przy niepoprawnym haśle oczekuje się odpowiedzi serwera z kodem statusu 400 oraz komunikatem błędu „wrong password”

Źródło: opracowanie własne.

```

Login
Success cases
✓ client login (63 ms)
✓ brand login (93 ms)
✓ developer login (81 ms)
Errors
✓ 400 user not found (6 ms)
✓ 400 wrong password (66 ms)

```

Rysunek 34 Wyniki testów scenariusza logowania. Źródło: opracowanie własne.

Tabela 3 Scenariusze testowe tokenów JWT

Nazwa testu	Kod	Opis
valid JWT token	<pre>const res = await request(routes) .get("/profileClient") .set("Authorization", `Bearer \${tokenClient}`); expect(res.statusCode).toBe(200);</pre>	Po udanym autentyfikacji tokenu oczekuje się odpowiedzi serwera z kodem statusu 200
401 no token provided	<pre>const res = await request(routes).get("/profileClient");</pre>	W przypadku braku przesłanego tokenu

	<pre>expect(res.statusCode).toBe(401); expect(res.body.error).toBe("No token provided");</pre>	serwer powinien zwrócić kod statusu 401 wraz z komunikatem „No token provided”.
403 invalid token	<pre>const res = await request(routes) .get("/profileClient") .set("Authorization", "Bearer BADTOKEN"); expect(res.statusCode).toBe(403); expect(res.body.error).toBe("Invalid token");</pre>	W przypadku przesłania wadliwego tokenu serwer powinien zwrócić kod statusu 403 wraz z komunikatem „Invalid token”.
404 Client not found	<pre>const wrongToken = jwt.sign({ id: 999, gender: "F" }, process.env.JWT_SECRET_TOKEN, { expiresIn: "1h" }); const res = await request(routes) .get("/profileClient") .set("Authorization", `Bearer \${wrongToken}`); expect(res.statusCode).toBe(404); expect(res.body.error).toBe("User not found");</pre>	W przypadku niezalezienia użytkownika przez serwer, oczekuje się statusu 404 wraz z komunikatem „User not found”.

Źródło: opracowanie własne.

```
JWT authentication
Sucess cases
  ✓ valid JWT (8 ms)
Errors
  ✓ 401 no token provided (4 ms)
  ✓ 403 invalid token (4 ms)
  ✓ 404 Client not found (10 ms)
```

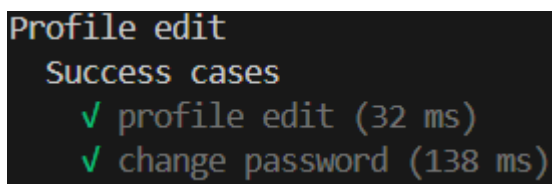
Rysunek 35 Wyniki testów walidacji tokenów JWT. Źródło: opracowanie własne.

Tabela 4 Scenariusze testowe edycji profilu

Nazwa testu	Kod	Opis
profile edit	<pre>const profileReq = await request(routes) .get("/profileClient") .set("Authorization", `Bearer \${tokenClient}`); expect(profileReq.statusCode).toBe(200); const userID = profileReq.body.userID; expect(userID).toBeDefined(); const clientEdit = { name: "newName", email: "newEmail@email.com" } const req = await request(routes) .post('/editProfile/'+userID) .send(clientEdit) .set("Authorization", `Bearer \${tokenClient}`); expect(req.statusCode).toBe(200);</pre>	Po udanym edycji profilu, oczekiwanym rezultatem jest odpowiedź serwera z kodem statusu 200.
change password	<pre>const profileReq = await request(routes) .get("/profileClient") .set("Authorization", `Bearer \${tokenClient}`); expect(profileReq.statusCode).toBe(200); const userID = profileReq.body.userID; expect(userID).toBeDefined(); const password = { password: 'clientToken', newPassword: 'newPassword' } const req = await request(routes) .post('/changePassword/'+userID)</pre>	Po udanym edycji hasła, oczekiwanym rezultatem jest odpowiedź serwera z kodem statusu 200.

	<pre> .send(password) .set("Authorization", `Bearer \${tokenClient}`); expect(req.statusCode).toBe(200); </pre>	
--	---	--

Źródło: opracowanie własne.



Rysunek 36 Wyniki testów edycji danych konta. Źródło: opracowanie własne.

Tabela 5 Scenariusze testowe przykładowe operacje PUT

Nazwa testu	Kod	Opis
put participation	<pre> const participation = { clientID : clientID, eventID : 1 } const req = await request(routes) .put('/putParticipation') .send(participation) .set("Authorization", `Bearer \${tokenBrand}`); expect(req.statusCode).toBe(200); </pre>	Po udanym wstawieniu udziału, oczekiwanym rezultatem jest odpowiedź serwera z kodem statusu 200.
put product	<pre> const product = { brandID: brandId, name: 'mockShoes', description: 'great shoes', price: 199, category: 'obuwie', imgURL: 'picture.jpg', } const req = await request(routes) </pre>	Po udanym wstawieniu produktu, oczekiwanym rezultatem jest odpowiedź serwera z kodem statusu 200.

	<pre> .put('/putProduct') .send(product) .set("Authorization", `Bearer \${tokenBrand}`); expect(req.statusCode).toBe(200); </pre>	
--	---	--

Źródło: opracowanie własne.

```

PUT commands
✓ put event (32 ms)
✓ put participation (36 ms)
✓ put product (37 ms)
✓ put product variant (24 ms)
✓ put order (36 ms)
✓ put review (22 ms)
✓ put article (18 ms)
✓ put comment (17 ms)

```

Rysunek 37 Wyniki testów operacji PUT. Źródło: opracowanie własne.

Tabela 6 Scenariusze testowe przykładowych operacje GET

Nazwa testu	Kod	Opis
client get orders	<pre> const req = await request(routes) .get('/getClientParticipation/'+clientID) .set("Authorization", `Bearer \${tokenClient}`); expect(req.statusCode).toBe(200); </pre>	Oczekiwany rezultatem jest pomyślne zwrócenie zamówień klienta wraz z kodem statusu 200.
Developer get articles	<pre> const req = await request(routes) .get('/getDevArticles/'+developerID) .set("Authorization", `Bearer \${tokenDeveloper}`); expect(req.statusCode).toBe(200); </pre>	Oczekiwany rezultatem jest pomyślne zwrócenie listy artykułów wraz z kodem statusu 200.

Źródło: opracowanie własne.

```

GET commands
✓ client get orders (21 ms)
✓ client get participations (17 ms)
✓ brand get product (18 ms)
✓ brand get orders (19 ms)
✓ brand get events (15 ms)
✓ Developer get Clients (7 ms)
✓ Developer get brands (7 ms)
✓ Developer get articles (17 ms)

```

Rysunek 38 Wyniki testów operacji GET. Źródło: opracowanie własne

Tabela 7 Scenariusze testowe integracji modułów

Nazwa testu	Kod	Opis
400 past event	<pre> const participation = { clientID : clientID, eventID : 2 } const req = await request(routes) .put('/putParticipation') .send(participation) .set("Authorization", `Bearer \${tokenClient}`); expect(req.statusCode).toBe(400); expect(req.body.error).toBe("too late to sign up"); </pre>	Sprawdza, czy użytkownik nie może zapisać się na wydarzenie, które już się odbyło. Oczekiwanym rezultatem jest kod 400 i komunikat „too late to sign up”.
400 already signed up for	<pre> const participation = { clientID : clientID, eventID : 1 } const req = await request(routes) .put('/putParticipation') .send(participation) </pre>	Weryfikuje, czy klient może zapisać się ponownie na to samo wydarzenie. Serwer powinien zwrócić kod 400 i komunikat

	<pre> .set("Authorization", `Bearer \${tokenClient}`); expect(req.statusCode).toBe(400); expect(req.body.error).toBe("already signed up"); </pre>	„already signed up”.
400 participations limit reached	<pre> const participation = { clientID : clientID, eventID : 1 } const req = await request(routes) .put('/putParticipation') .send(participation) .set("Authorization", `Bearer \${tokenClient2}`); expect(req.statusCode).toBe(400); expect(req.body.error).toBe("limit reached"); </pre>	Testuje sytuację, gdy osiągnięto limit uczestników wydarzenia. Oczekiwanym wynikiem jest kod 400 i komunikat „limit reached”.
400 negative price	<pre> const product = { brandID: brandId, name: 'mockShoes', description: 'great shoes', price: -200, category: 'obuwie', imgURL: 'picture.jpg', } const req = await request(routes) .put('/putProduct') .send(product) .set("Authorization", `Bearer \${tokenBrand}`); expect(req.statusCode).toBe(400); expect(req.body.error).toBe('price cant be negative'); </pre>	Sprawdza, czy aplikacja odrzuca próbę dodania produktu z ujemną ceną. Oczekiwanym rezultatem jest kod 400 i komunikat „price cant be negative”.

400 one-size collision	<pre>const productVariant = { productID: 1, size: 'M', stock: 10, } const req = await request(routes) .put('/putProductVariant') .send(productVariant) .set("Authorization", `Bearer \${tokenBrand}`); expect(req.statusCode).toBe(400); expect(req.body.error).toBe('Cannot add sized variant');</pre>	Weryfikuje, czy nie można dodać wariant rozmiarowy „sized” dla produktu już istniejącego jako „one-size”. Oczekiwany kod to 400, komunikat: „Cannot add sized variant”.
400 negative stock	<pre>const productVariant = { productID: 1, size: 'one-size', stock: -10, } const req = await request(routes) .put('/putProductVariant') .send(productVariant) .set("Authorization", `Bearer \${tokenBrand}`); expect(req.statusCode).toBe(400); expect(req.body.error).toBe('Stock cant be negative');</pre>	Sprawdza, czy aplikacja odrzuca dodanie wariantu produktu z ujemnym stanem magazynowym. Oczekiwany kod: 400, komunikat „Stock cant be negative”.
400 cant review uncompleted order	<pre>const review = { clientID: clientID, productID: 2, rating: 4, comment: 'super', } const req = await request(routes)</pre>	Testuje, czy klient może ocenić produkt przed zakończeniem zamówienia. Oczekiwany wynik jest kod

	<pre> .put('/putReview') .send(review) .set("Authorization", `Bearer \${tokenClient}`); expect(req.statusCode).toBe(400); expect(req.body.error).toBe('cant review uncompleted orders'); </pre>	400 i komunikat „cant review uncompleted orders”.
400 cant review without order	<pre> const review = { clientID: clientID, productID: 3, rating: 4, comment: 'super', } const req = await request(routes) .put('/putReview') .send(review) .set("Authorization", `Bearer \${tokenClient}`); expect(req.statusCode).toBe(400); expect(req.body.error).toBe('you cant review without an order') </pre>	Sprawdza, czy klient może dodać recenzję produktu, którego nie zamówił. Oczekiwany kod: 400, komunikat „you cant review without an order”.
400 article title or content is empty	<pre> const article = { developerID: developerID, title: "", content: "", imgUrl: 'img.jpg', } const req = await request(routes) .put('/putArticle') .send(article) .set("Authorization", `Bearer \${tokenDeveloper}`); expect(req.statusCode).toBe(400); </pre>	Weryfikuje, czy system odrzuca próbę dodania artykułu bez tytułu lub treści. Oczekiwany rezultat jest kod 400 i komunikat „title and content cant be empty”.

	<code>expect(req.body.error).toBe('title and content cant be empty')</code>	
400 comment is empty	<pre>const comment = { clientID: clientID, articleID: 1, content: "", } const req = await request(routes) .put('/putComment') .send(comment) .set("Authorization", `Bearer \${tokenClient}`); expect(req.statusCode).toBe(400); expect(req.body.error).toBe('content is empty');</pre>	Testuje, czy klient może dodać komentarz bez treści. Oczekiwany kod: 400 i komunikat „content is empty”.

Źródło: opracowanie własne.

Integration tests
✓ 400 past event (15 ms)
✓ 400 already signed up for (18 ms)
✓ 400 participations limit reached (20 ms)
✓ 400 negative price (11 ms)
✓ 400 one-size collision (6 ms)
✓ 400 negative stock (4 ms)
✓ 400 cant review uncompleted order (14 ms)
✓ 400 cant review without order (15 ms)
✓ 400 article title or content is empty (10 ms)
✓ 400 comment is empty (9 ms)

Rysunek 39 Wyniki testów integracyjnych. Źródło: opracowanie własne

Tabela 8 Scenariusze testowe przykładowe operacje POST

Nazwa testu	Kod	Opis
post participation	<pre>const req = await request(routes) .post('/postParticipation/'+1)</pre>	Po pomyślnej edycji udziału oczekiwanym

	<pre> .send({placement:1}) .set("Authorization", `Bearer \${tokenBrand}`); expect(req.statusCode).toBe(200); </pre>	rezultatem jest odpowiedź serwera z kodem statusu 200.
post article	<pre> const req = await request(routes) .post('/postProductVariant/'+1) .send({ title: 'NewTitle', content: 'newContent', imgURL: 'picture.jpg', }) .set("Authorization", `Bearer \${tokenBrand}`); expect(req.statusCode).toBe(200); </pre>	Po pomyślnej edycji aktualności oczekiwanym rezultatem jest odpowiedź serwera z kodem statusu 200.

Źródło: opracowanie własne.

```

POST commands
✓ post participation (6 ms)
✓ post event (8 ms)
✓ post product (57 ms)
✓ post variant (5 ms)
✓ post order (5 ms)
✓ post article (4 ms)

```

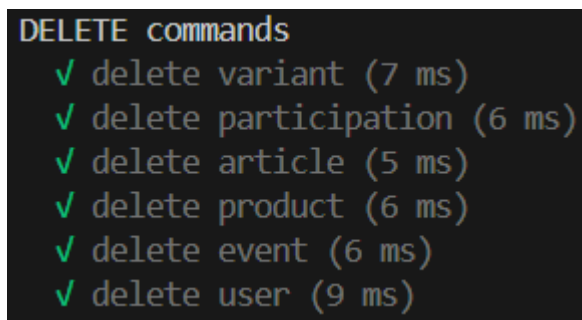
Rysunek 40 Wyniki testów operacji POST. Źródło: opracowanie własne

Tabela 9 Scenariusze testowe operacji DELETE

Nazwa testu	Kod	Opis
delete variant	<pre> const req = await request(routes) .delete('/deleteVariant/1') .set("Authorization", `Bearer \${tokenBrand}`); expect(req.statusCode).toBe(200); </pre>	Po udanym usunięciu wariantu produktu oczekiwanym rezultatem jest odpowiedź serwera z kodem statusu 200.

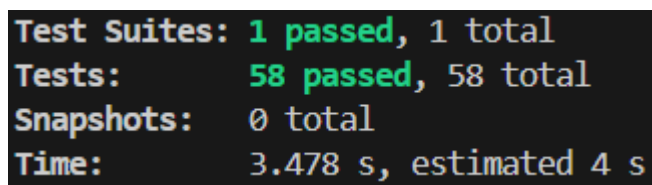
delete user	<pre>const req = await request(routes) .delete('/deleteUser/1') .set("Authorization", `Bearer     \${tokenDeveloper}`); expect(req.statusCode).toBe(200);</pre>	Po udanym usunięciu użytkownika oczekiwanym rezultatem jest odpowiedź serwera z kodem statusu 200.
-------------	---	--

Źródło: opracowanie własne.



Rysunek 41 Wyniki testów operacji DELETE. Źródło: opracowanie własne

Przeprowadzone testy jednostkowe oraz integracyjne wykazały poprawne działanie wszystkich sprawdzanych funkcjonalności aplikacji. W ramach jednego zestawu testów uruchomiono łącznie 58 testów, z których wszystkie zakończyły się powodzeniem, co ilustruje wynik przedstawiony na Rysunek 42.



Rysunek 42 Podsumowanie wyników przeprowadzonych testów. Źródło: opracowanie własne.

Wyniki te potwierdzają, że implementowane moduły backendu spełniają założone wymagania funkcjonalne, a aplikacja działa zgodnie z oczekiwaniami w przewidzianych scenariuszach testowych.

5.2. Testy end-to-end aplikacji z wykorzystaniem Cypress

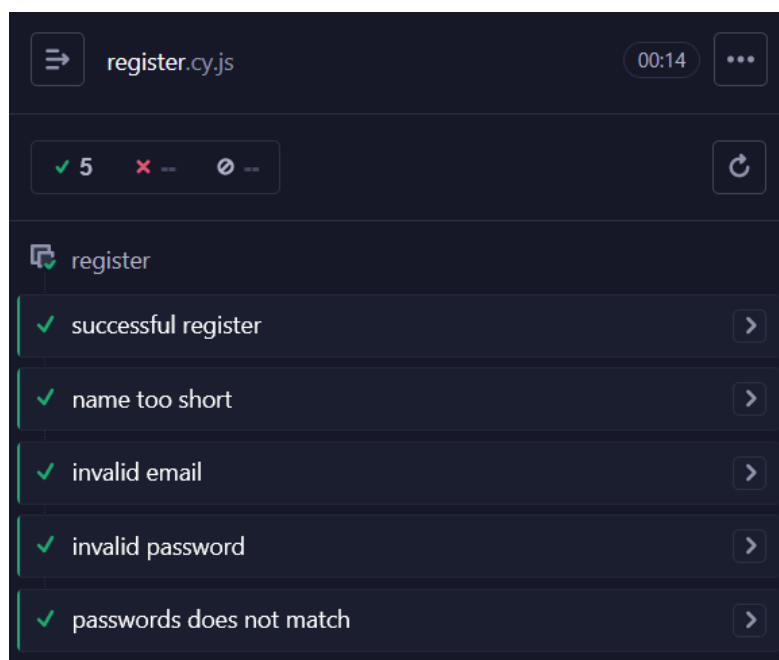
Tabela 10 Scenariusze testowe logowania - frontend

Nazwa testu	Kod	Opis
-------------	-----	------

successful register	<pre> cy.visit('localhost:3000') cy.contains("button","zaloguj się").click(); cy.get("#register-link").click(); cy.get('#name').clear().type('mockUser') cy.get('#email').clear().type('mockUser@mock.com') cy.get('#password').clear().type('mockUser') cy.get('#re-password').clear().type('mockUser') cy.get('#birthdate').clear().type('2002-11-13') cy.get('#selectGender').select('M') cy.contains("button","Rejestruj").click(); cy.url().should("include","/login") </pre>	Sprawdza poprawny proces rejestracji nowego użytkownika. Po wprowadzeniu prawidłowych danych użytkownik zostaje przekierowany do strony logowania.
name too short	<pre> cy.visit('localhost:3000') cy.contains("button","zaloguj się").click(); cy.get("#register-link").click(); cy.get('#name').clear().type('na') cy.get('#email').clear().type('newUser@mock.com') cy.get('#password').clear().type('mockUser') cy.get('#re-password').clear().type('mockUser') cy.get('#birthdate').clear().type('2002-11-13') cy.get('#selectGender').select('M') cy.contains("button","Rejestruj").click(); cy.contains("Nazwa musi mieć co najmniej 3 znaki") </pre>	Weryfikuje walidację pola nazwy użytkownika. W przypadku zbyt krótkiej nazwy wyświetlany jest komunikat o błędzie.
invalid email	<pre> cy.visit('localhost:3000') cy.contains("button","zaloguj się").click(); cy.get("#register-link").click(); cy.get('#name').clear().type('newName') cy.get('#email').clear().type('newUsercom') cy.get('#password').clear().type('mockUser') cy.get('#re-password').clear().type('mockUser') </pre>	Sprawdza, czy system wykrywa niepoprawny format adresu e-mail i informuje

	cy.get('#birthdate').clear().type('2002-11-13') cy.get('#selectGender').select('M') cy.contains("button", "Rejestruj").click(); cy.contains("Niepoprawny e-mail")	użytkownika o błędzie.
invalid password	cy.visit('localhost:3000') cy.contains("button", "zaloguj się").click(); cy.get("#register-link").click(); cy.get('#name').clear().type('newName') cy.get('#email').clear().type('newUser@mock.com') cy.get('#password').clear().type('mock') cy.get('#re-password').clear().type('mock') cy.get('#birthdate').clear().type('2002-11-13') cy.get('#selectGender').select('M') cy.contains("button", "Rejestruj").click(); cy.contains("hasło zbyt słabe")	Testuje walidację hasła. Przy zbyt słabym hasle rejestracja zostaje zablokowana.
passwords does not match	cy.visit('localhost:3000') cy.contains("button", "zaloguj się").click(); cy.get("#register-link").click(); cy.get('#name').clear().type('newName') cy.get('#email').clear().type('newUser@mock.com') cy.get('#password').clear().type('mockBrand') cy.get('#re-password').clear().type('mockBran') cy.get('#birthdate').clear().type('2002-11-13') cy.get('#selectGender').select('M') cy.contains("button", "Rejestruj").click(); cy.contains("hasła nie identyczne")	Weryfikuje, czy system poprawnie wykrywa niezgodność haseł i uniemożliwia rejestrację.

Źródło: opracowanie własne.



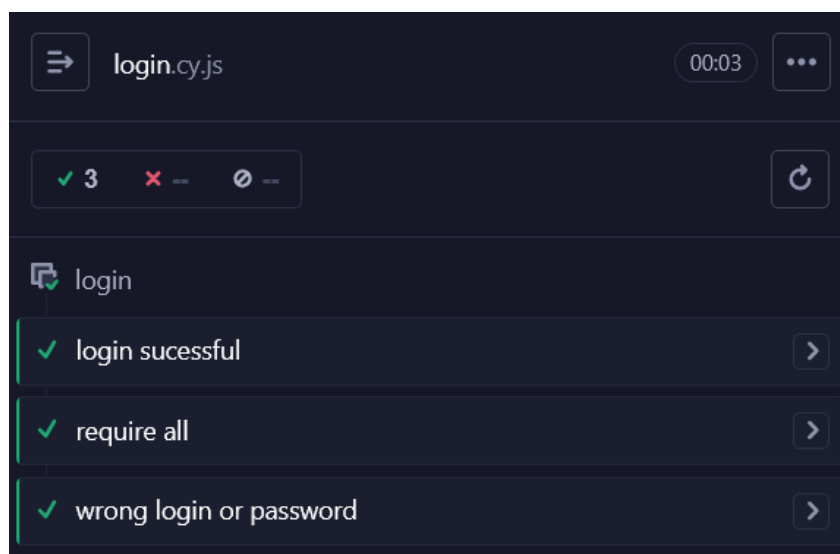
Rysunek 43 Wyniki testów scenariuszy rejestrowania - frontend. Źródło: opracowanie własne.

Tabela 11 Scenariusze testowe logowania - frontend

Nazwa testu	Kod	Opis
login sucessful	<pre>cy.visit('localhost:3000') cy.contains("button","zaloguj się").click(); cy.get('#email').clear().type('mockUser@mock.com') cy.get('#password').clear().type('mockUser') cy.get("#login").click();</pre>	Sprawdza poprawny proces logowania użytkownika przy użyciu prawidłowych danych uwierzytelniających.
require all	<pre>it('require all', () => { cy.visit('localhost:3000') cy.contains("button","zaloguj się").click(); cy.get("#login").click(); cy.contains("Wszystkie pola są wymagane") })</pre>	Weryfikuje, czy system wymaga uzupełnienia wszystkich pól formularza logowania.

wrong login or password	<pre> cy.visit('localhost:3000') cy.contains("button","zaloguj się").click(); cy.get('#email').clear().type('mockUser@moc.com') cy.get('#password').clear().type('mockUse') cy.get("#login").click(); cy.contains("Niepoprawny e_mail lub hasło") </pre>	Sprawdza reakcję systemu na podanie niepoprawnego adresu e-mail lub hasła.
-------------------------	---	--

Źródło: opracowanie własne.



Rysunek 44 Wyniki testów scenariuszy logowania - frontend. Źródło: opracowanie własne.

Tabela 12 Scenariusze testowe dodawania wydarzenia

Nazwa testu	Kod	Opis
add successful	<pre> cy.brandLogin(); cy.visit("http://localhost:3000/brand"); cy.get("#products").click(); cy.get("#new").click(); cy.get('#name-input').clear().type('shoes') cy.get('#price-input').clear().type('99') cy.get('#description-input').clear().type('great shoes') cy.get('#category-input').select('obuwie') </pre>	Sprawdza możliwość poprawnego dodania nowego produktu przez konto firmy.

	<pre>cy.get('#url-input').clear().type('pic.jpg') cy.contains("button", "akceptuj").click();</pre>	
invalid name	<pre>cy.brandLogin(); cy.visit("http://localhost:3000/brand"); cy.get("#products").click(); cy.get("#new").click(); cy.get('#price-input').clear().type('99') cy.get('#description- input').clear().type('great shoes') cy.get('#category-input').select('obuwie') cy.get('#url-input').clear().type('pic.jpg') cy.contains("button", "akceptuj").click(); cy.contains("wprowadź nazwę")</pre>	Weryfikuje, czy system wymaga podania nazwy produktu.
invalid price	<pre>cy.brandLogin(); cy.visit("http://localhost:3000/brand"); cy.get("#products").click(); cy.get("#new").click(); cy.get('#name- input').clear().type('shoes') cy.get('#description- input').clear().type('great shoes') cy.get('#category-input').select('obuwie') cy.get('#url-input').clear().type('pic.jpg') cy.contains("button", "akceptuj").click(); cy.contains("Niepoprawna cena")</pre>	Sprawdza walidację ceny produktu.
invalid description	<pre>cy.brandLogin(); cy.visit("http://localhost:3000/brand"); cy.get("#products").click(); cy.get("#new").click(); cy.get('#name- input').clear().type('shoes') cy.get('#price-input').clear().type('99') cy.get('#category-input').select('obuwie')</pre>	Testuje reakcję systemu na brak opisu produktu.

	<pre> cy.get('#url-input').clear().type('pic.jpg') cy.contains("button","akceptuj").click(); cy.contains("wprowadź opis") </pre>	
invalid category	<pre> cy.brandLogin(); cy.visit("http://localhost:3000/brand"); cy.get("#products").click(); cy.get("#new").click(); cy.get('#name- input').clear().type('shoes') cy.get('#price-input').clear().type('99') cy.get('#description- input').clear().type('great shoes') cy.get('#url-input').clear().type('pic.jpg') cy.contains("button","akceptuj").click(); cy.contains("wybierz kategorię") </pre>	Weryfikuje konieczność wyboru kategorii produktu.
invalid image-url	<pre> cy.brandLogin(); cy.visit("http://localhost:3000/brand"); cy.get("#products").click(); cy.get("#new").click(); cy.get('#name- input').clear().type('shoes') cy.get('#price-input').clear().type('99') cy.get('#description- input').clear().type('great shoes') cy.get('#category-input').select('obuwie') cy.contains("button","akceptuj").click(); cy.contains("Wprowadź link obrazka") </pre>	Sprawdza, czy brak adresu URL obrazka powoduje błąd walidacji.

Źródło: opracowanie własne.

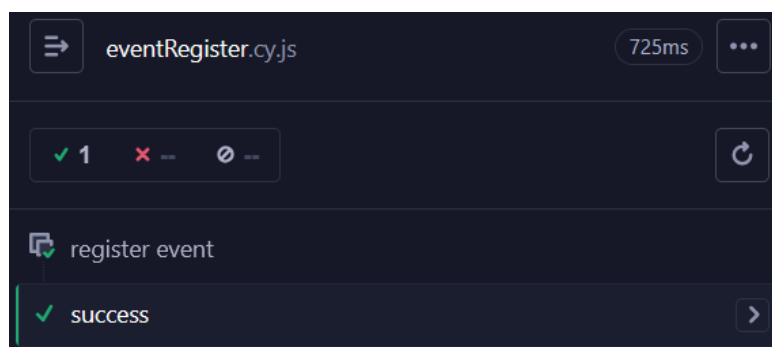
✓ add event successful	>
✓ invalid name	>
✓ invalid description	>
✓ invalid city	>
✓ invalid address	>
✓ invalid capacity	>
✓ invalid image-url	>

Rysunek 45 Wyniki testów scenariuszy dodawania wydarzenia. Źródło: opracowanie własne.

Tabela 13 Scenariusze testowe udziału w wydarzeniu

Nazwa testu	Kod	Opis
Join event	<pre> cy.clientLogin() cy.visit('localhost:3000') cy.get("#events").click(); cy.contains("h3","WLB CRUX") .click() cy.contains("button","zapisz się") .click() cy.contains("zapisaleś się na turniej") </pre>	Sprawdza, czy zalogowany klient może zapisać się na dostępne wydarzenie. Po zapisaniu wyświetlany jest komunikat potwierdzający.

Źródło: opracowanie własne.



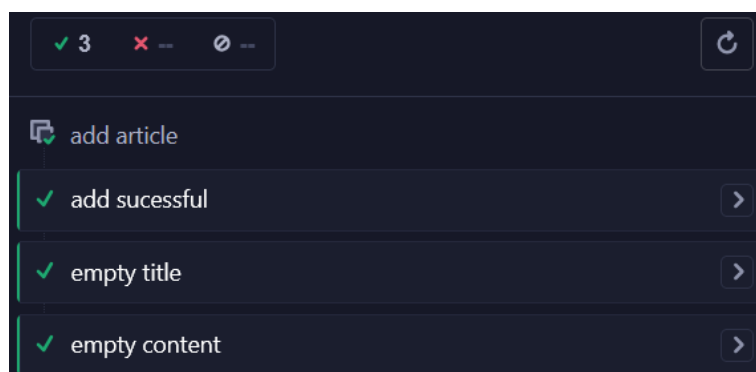
Rysunek 46 Wyniki testu scenariusza zapisu na wydarzenie. Źródło: opracowanie własne.

Tabela 14 Scenariusze testowe dodania artykułu

Nazwa testu	Kod	Opis
add sucessful	<pre> cy.developerLogin(); cy.visit("http://localhost:3000/emp"); cy.get("#articles").click(); cy.get("#new").click(); cy.get('#title- input').clear().type('Wyniki finału turnieju w Seoulu') cy.get('#content-input').clear().type('cos tam cos tam') cy.get('#url-input').clear().type('pic.jpg') cy.contains("button","akceptuj").click(); </pre>	Sprawdza możliwość poprawnego dodania nowego artykułu przez pracownika systemu.
empty title	<pre> cy.developerLogin(); cy.visit("http://localhost:3000/emp"); cy.get("#articles").click(); cy.get("#new").click(); cy.get('#content-input').clear().type('cos tam cos tam') cy.get('#url-input').clear().type('pic.jpg') cy.contains("button","akceptuj").click(); cy.contains("tytuł nie może być pusty") </pre>	Weryfikuje, czy system pozwala na dodanie artykułu bez tytułu.

empty content	<pre> cy.developerLogin(); cy.visit("http://localhost:3000/emp"); cy.get("#articles").click(); cy.get("#new").click(); cy.get('#title- input').clear().type('Wyniki finału turnieju w Seoulu') cy.get('#url-input').clear().type('pic.jpg') cy.contains("button","akceptuj").click(); cy.contains("treść nie może być pusty") </pre>	Sprawdza walidację treści artykułu – brak treści powoduje błąd.
---------------	--	---

Źródło: opracowanie własne.



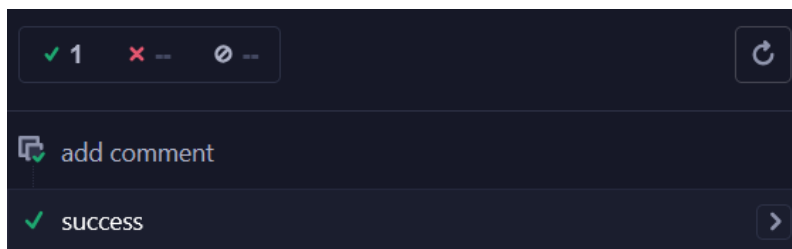
Rysunek 47 Wyniki testu scenariuszy dodania artykułu. Źródło: opracowanie własne.

Tabela 15 Scenariusze testowe komentowania artykułu

Nazwa testu	Kod	Opis
Comment successful	<pre> cy.clientLogin() cy.visit('localhost:3000') cy.get('#articles').click() cy.contains('h3',"Wyniki finału turnieju w Seoulu").click() cy.get("#add- comment").clear().type("super artykuł") </pre>	Sprawdza możliwość dodania komentarza przez użytkownika pod opublikowanym artykułem.

	cy.contains("button","dodaj komentarz").click()	
--	---	--

Źródło: opracowanie własne.



Rysunek 48 Wyniki testu scenariusza komentowania artykułu. Źródło: opracowanie własne.

Testy end-to-end zostały zaprojektowane w taki sposób, aby odwzorować rzeczywiste scenariusze użytkowania aplikacji webowej przez różnych aktorów systemu. Skupiono się na kluczowych procesach biznesowych, takich jak rejestracja i logowanie użytkowników, dodawanie wydarzeń przez firmy, zapisy klientów na wydarzenia oraz publikowanie artykułów i komentarzy.

Przeprowadzone testy umożliwiły weryfikację poprawnej współpracy pomiędzy warstwą frontendową a backendową aplikacji. Scenariusze testowe potwierdziły prawidłowe działanie interakcji pomiędzy klientami, firmami oraz deweloperami, co odzwierciedla rzeczywisty sposób korzystania z systemu.

Uzyskane wyniki testów end-to-end potwierdzają, że aplikacja działa zgodnie z założeniami projektowymi, poprawną integrację frontendu z backendem oraz prawidłowe działanie podstawowych procesów biznesowych.

6. Możliwości rozwoju aplikacji

6.1. Rozwój i optymalizacja zastosowanych technologii

Jednym z obszarów wymagających dalszego rozwoju jest sposób obsługi zasobów graficznych w aplikacji. Aktualnie obrazy produktów dodawane są na podstawie zewnętrznych adresów URL, co upraszcza implementację, lecz wiąże się z potencjalnymi zagrożeniami bezpieczeństwa. Poniższy fragment kodu przedstawia aktualnie zastosowany mechanizm przypisywania adresu URL obrazu na podstawie danych wprowadzanych przez użytkownika (Rysunek 49).

```
<input type="text" defaultValue={offer.imgURL} id="url-input" onChange={e => {  
  offer.imgURL = e.target.value;  
  document.getElementById("box-art").src = e.target.value;  
}}/><br/>  
  
<img src={offer.imgURL} alt="box art" className="large-product-img" id="box-art"/><br/>
```

Rysunek 49 Aktualna obsługa zasobów graficznych. Źródło: opracowanie własne.

Aplikacja nie ma kontroli nad zawartością znajdującą się pod wskazanym adresem, co może prowadzić do wyświetlania niepożądanych treści lub zasobów zmienionych po czasie. Rozwiązanie to zostało przyjęte na etapie prototypowania aplikacji, jednak w dalszym rozwoju systemu wymaga zastąpienia bezpieczniejszym mechanizmem obsługi plików.

W przyszłości zalecanym rozwiązaniem byłoby wprowadzenie mechanizmu przesyłania plików graficznych bezpośrednio na serwer lub do dedykowanego systemu przechowywania plików, wraz z walidacją typu, rozmiaru oraz formatu obrazu.

Drugą z technologii, która może zostać istotnie rozwinięta i zoptymalizowana, jest mechanizm uwierzytelniania oparty na tokenach JWT (JSON Web Token). W obecnej wersji systemu JWT wykorzystywany jest do weryfikacji tożsamości użytkownika oraz kontroli dostępu do chronionych zasobów aplikacji, co pozwala na bezstanową i wydajną autoryzację po stronie serwera.

Pomimo licznych zalet tego rozwiązania, takich jak prostota implementacji, skalowalność oraz brak konieczności przechowywania sesji po stronie serwera, istnieje szereg obszarów, które mogą zostać ulepszone w przyszłych wersjach systemu. Jednym z nich jest zwiększenie poziomu bezpieczeństwa poprzez zastosowanie krótkiego czasu życia tokenów

dostępowych (access token) oraz wprowadzenie mechanizmu tokenów odświeżających (refresh token) [19]. Takie podejście ogranicza ryzyko nieautoryzowanego dostępu w przypadku przejęcia tokenu.

Dodatkowo możliwe jest rozszerzenie systemu o mechanizmy unieważniania tokenów (token blacklist), co pozwoliłoby na natychmiastowe wylogowanie użytkownika, na przykład po zmianie hasła lub wykryciu podejrzanej aktywności. W obecnym rozwiązaniu token zachowuje ważność do momentu wygaśnięcia, co w niektórych scenariuszach może stanowić potencjalne zagrożenie.

6.2. Rozszerzenie interakcji między użytkownikami

Jednym z kluczowych kierunków dalszego rozwoju systemu jest rozbudowa mechanizmów interakcji pomiędzy użytkownikami platformy. Obecnie aplikacja umożliwia podstawowe formy komunikacji pośredniej, takie jak udział klientów w wydarzeniach, wystawianie ocen po zakończonych zamówieniach oraz komentowanie artykułów publikowanych przez deweloperów. Funkcjonalności te spełniają swoją rolę informacyjną, jednak mogą zostać znacząco rozszerzone w celu zwiększenia zaangażowania użytkowników.

Naturalnym krokiem rozwojowym byłoby wprowadzenie systemu bezpośredniej komunikacji, na przykład w formie wiadomości prywatnych pomiędzy klientem a firmą. Pozwoliłoby to na szybsze wyjaśnianie szczegółów dotyczących zamówień, produktów lub wydarzeń, bez konieczności korzystania z zewnętrznych kanałów komunikacji. Taki mechanizm mógłby być dodatkowo powiązany z konkretnym zamówieniem lub wydarzeniem, co zwiększyłoby jego przejrzystość.

Kolejnym możliwym rozszerzeniem jest rozwój funkcji społecznościowych, takich jak reakcje na artykuły i komentarze (np. polubienia), a także możliwość odpowiedzi na komentarze. Pozwoliłoby to na tworzenie bardziej rozbudowanych dyskusji wokół publikowanych treści i zwiększyłoby aktywność użytkowników w module aktualności.

W kontekście wydarzeń interesującym rozwiązaniem byłoby dodanie profili uczestników turniejów wraz z widocznymi wynikami i historią udziału w wydarzeniach. Umożliwiłoby to budowanie rankingów użytkowników oraz zwiększyłoby element rywalizacji, co jest szczególnie istotne w aplikacjach związanych z wydarzeniami i turniejami.

Dodatkowo system mógłby zostać wzbogacony o mechanizmy powiadomień, informujące użytkowników o istotnych zdarzeniach, takich jak zmiana statusu zamówienia,

rozpoczęcie wydarzenia, odpowiedź na komentarz czy publikacja nowego artykułu. Powiadomienia te mogłyby być realizowane zarówno w obrębie aplikacji, jak i za pomocą poczty elektronicznej.

6.3. Wdrożenie aplikacji mobilnej

Istotnym kierunkiem rozwoju systemu jest stworzenie dedykowanej aplikacji mobilnej. Ze względu na charakter platformy oraz grupę docelową użytkowników, aplikacja mobilna mogłaby znacząco zwiększyć dostępność i wygodę korzystania z systemu. Urządzenia mobilne są naturalnym narzędziem towarzyszącym użytkownikom w trakcie aktywności sportowych, co czyni je idealną platformą dla tego typu rozwiązania.

Aplikacja mobilna mogłaby korzystać z już istniejącego API, co pozwoliłoby na ponowne wykorzystanie logiki biznesowej systemu. Dodatkowo umożliwiłaby implementację funkcji charakterystycznych dla urządzeń mobilnych, takich jak powiadomienia push czy dostęp offline.

6.4. Moduł treningowy

Kolejnym rozszerzeniem systemu, szczególnie dobrze wpisującym się w koncepcję aplikacji mobilnej, jest moduł treningowy. Moduł ten mógłby umożliwiać użytkownikom planowanie i rejestrowanie treningów, śledzenie postępów oraz dostęp do gotowych planów treningowych. Na rynku istnieje wiele aplikacji treningowych, co potwierdza zapotrzebowanie na tego typu rozwiązania, zwłaszcza w formie mobilnej, która może towarzyszyć użytkownikowi bezpośrednio podczas treningu. Moduł treningowy mógłby być powiązany z wydarzeniami oraz rankingami, umożliwiając użytkownikom lepsze przygotowanie do zawodów oraz analizę własnych wyników.

7. Podsumowanie

W ramach niniejszej pracy zaprojektowano oraz zaimplementowano aplikację webową dedykowaną społeczności wspinaczkowej, której głównym zadaniem jest centralizacja informacji oraz usług związanych z tą dyscypliną sportu. Zrealizowany system spełnia założone wymagania funkcjonalne i нефункционалне, umożliwiając obsługę różnych ról użytkowników oraz zapewniając poprawną i bezpieczną realizację kluczowych procesów biznesowych.

Aplikacja pozwala na kompleksową obsługę scenariuszy obejmujących rejestrację i logowanie użytkowników, zarządzanie wydarzeniami, publikację artykułów, obsługę komentarzy oraz realizację procesów zakupowych. Szczególną uwagę poświęcono poprawnemu modelowaniu relacji pomiędzy obiektami systemu oraz kontroli uprawnień, co umożliwiło jednoznaczne rozdzielenie odpowiedzialności pomiędzy poszczególne role użytkowników, takie jak klient, firma, deweloper oraz administrator.

W projekcie uwzględniono wytyczne WCAG (Web Content Accessibility Guidelines), których celem jest zapewnienie możliwie szerokiego dostępu do treści cyfrowych dla użytkowników o zróżnicowanych potrzebach. W szczególności uwzględniono cztery podstawowe zasady dostępności: postrzegalność (Perceivable), funkcjonalność (Operable), zrozumiałość (Understandable) oraz solidność techniczną (Robust) [20].

W zakresie zasady Perceivable, zgodnie z wytyczną 1.4.1 „Use of Color”, informacje o błędach formularzy nie są przekazywane wyłącznie za pomocą koloru [20]. Błędy oznaczane są kolorem czerwonym i każdorazowo wyświetlany jest komunikat tekstowy informujący o przyczynie błędu. Dodatkowo interfejs został zaprojektowany z zachowaniem odpowiedniego kontrastu pomiędzy tekstem a tłem, a elementy formularzy posiadają czytelne i jednoznaczne etykiety

Zasada Operable została uwzględniona m.in. poprzez stosowanie jednoznacznych tytułów podstron zgodnie z wytyczną 2.4.2 „Page Titled”, co ułatwia orientację w strukturze aplikacji [20]. Elementy formularzy zostały opatrzone etykietami (label), a nawigacja aplikacji zachowuje spójność pomiędzy widokami, umożliwiając intuicyjne i przewidywalne korzystanie z systemu.

W kontekście zasady Understandable zastosowano czytelne treści interfejsu oraz jednoznaczne komunikaty walidacyjne. Błędy danych wejściowych są wykrywane i

przekazywane użytkownikowi w sposób zrozumiały, co pozwala na ich łatwą korektę i ogranicza ryzyko błędnej obsługi aplikacji.

Zasada Robust została spełniona poprzez poprawne definiowanie elementów interfejsu zgodnie z wytycznymi 4.1.1 „Parsing” oraz 4.1.2 „Name, Role, Value” [20]. Struktura dokumentów oparta na znacznikach HTML jest zgodna ze specyfikacją języka, zawiera kompletne znaczniki otwarcia i zamknięcia, poprawne zagnieżdżenia elementów oraz unikalne identyfikatory. Komponenty interaktywne posiadają jednoznacznie określone nazwy, role oraz stany.

Kolejnym istotnym elementem pracy była weryfikacja poprawności działania systemu poprzez zastosowanie testów automatycznych, w tym testów integracyjnych oraz end-to-end. Przygotowane scenariusze testowe potwierdziły prawidłową obsługę zarówno przypadków poprawnych, jak i sytuacji błędnych, takich jak brak autoryzacji, nieprawidłowe dane wejściowe czy naruszenie reguł biznesowych. Przeprowadzone testy pozwoliły również upewnić się, że wprowadzane zmiany nie powodują niepożądanych regresji w istniejących mechanizmach systemu.

Zastosowane rozwiązania architektoniczne oraz mechanizmy bezpieczeństwa, w szczególności uwierzytelnianie oparte na tokenach JWT, zapewniają bezpieczną komunikację pomiędzy warstwą kliencką a serwerową. System został zaprojektowany w sposób umożliwiający jego dalszą rozbudowę oraz łatwe wdrożenie w różnych środowiskach, co potwierdza jego uniwersalność i praktyczny charakter.

Podsumowując, zrealizowana aplikacja stanowi funkcjonalne i spójne narzędzie wspierające społeczność wspinaczkową, a przeprowadzone testy potwierdzają jej poprawne działanie oraz gotowość do dalszego rozwoju. Osiągnięte rezultaty dowodzą, że przyjęte założenia projektowe zostały skutecznie zrealizowane, a system spełnia wymagania stawiane nowoczesnym aplikacjom webowym.

Bibliografia

- [1] A. Tanenbaum, N. Feamster i D. Wetherall, COMPUTER NETWORKS, Harlow: Pearson, 2021.
- [2] IBM, „Architektura klient/serwer,” 03 03 2021. [Online]. Available: <https://www.ibm.com/docs/pl/informix-servers/12.10.0?topic=communication-clientserver-architecture>. [Data uzyskania dostępu: 24 12 2025].
- [3] A. Kapczyński i S. Smugowski, WSPÓŁCZESNE SYSTEMY INFORMATYCZNE I ICH ZASTOSOWANIA, Katowice: PTI, 2010.
- [4] MDN, „SPA (Single-page application),” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Glossary/SPA>. [Data uzyskania dostępu: 24 12 2025].
- [5] R. Wieruch, The Road to learn React, Berlin: wydanie własne, 2017.
- [6] AWS, „What’s the Difference Between Frontend and Backend in Application Development?,” [Online]. Available: <https://aws.amazon.com/compare/the-difference-between-frontend-and-backend/>. [Data uzyskania dostępu: 24 12 2025].
- [7] ExpressJS, „Express,” [Online]. Available: <https://expressjs.com/>. [Data uzyskania dostępu: 25 12 2025].
- [8] MDN, „JavaScript,” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>. [Data uzyskania dostępu: 25 12 2025].
- [9] Telkom University, „ReactJS: Definition, Features, Benefits, How it Works and Advantages,” 16 11 2023. [Online]. Available: <https://jakarta.telkomuniversity.ac.id/reactjs-definition-features-benefits-how-it-works-and-advantages/>. [Data uzyskania dostępu: 24 12 2025].
- [10] A. Dybel, Analiza bezpieczeństwa aplikacji Next.js, Kraków: Politechnika Krakowska, 2024.
- [11] MDN, „CSS: Cascading Style Sheets,” [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/CSS>. [Data uzyskania dostępu: 24 12 2025].
- [12] I18next, „Introduction | i18next documentation,” 11 11 2025. [Online]. Available: <https://www.i18next.com/>. [Data uzyskania dostępu: 12 01 2026].

- [13] NodeJS, „About Node.js,” [Online]. Available: <https://nodejs.org/en/about>. [Data uzyskania dostępu: 24 12 2025].
- [14] Oracle, „MySQL: Understanding What It Is and How It’s Used,” 29 08 2024. [Online]. Available: <https://www.oracle.com/mysql/what-is-mysql/>. [Data uzyskania dostępu: 25 12 2025].
- [15] MySQL, „MySQL,” [Online]. Available: <https://www.mysql.com/>. [Data uzyskania dostępu: 25 12 2025].
- [16] Sequelize, „Getting started,” 25 08 2025. [Online]. Available: <https://sequelize.org/docs/v6/getting-started/>. [Data uzyskania dostępu: 26 12 2025].
- [17] Cypress, „Why Cypress?,” 05 05 2025. [Online]. Available: <https://docs.cypress.io/app/get-started/why-cypress>. [Data uzyskania dostępu: 25 12 2025].
- [18] MDN, „Test Javascript with Jest,” [Online]. Available: <https://developer.mozilla.org/en-US/blog/test-javascript-with-jest-on-vultr/>. [Data uzyskania dostępu: 25 12 2025].
- [19] Auth0, „Refresh Tokens,” [Online]. Available: <https://auth0.com/docs/secure/tokens/refresh-tokens>. [Data uzyskania dostępu: 12 01 2026].
- [20] W3C, „How to Meet WCAG,” 22 08 2025. [Online]. Available: <https://www.w3.org/WAI/WCAG22/quickref/?versions=2.1>. [Data uzyskania dostępu: 12 01 2026].

Spis rysunków

Rysunek 1 Diagram przypadków użycia aplikacji wspinaczkowej. Źródło: opracowanie własne.....	15
Rysunek 2 Sposób inicjalizacji i uruchomienia aplikacji React. Źródło: https://create-react-app.dev/docs/getting-started/	18
Rysunek 3 Sposoby inicjalizacji biblioteki i18next. Źródło: https://www.i18next.com/overview/getting-started	18
Rysunek 4 Konfiguracja i18next z plikami tłumaczeń JSON. Źródło: opracowanie własne..	19
Rysunek 5 Plik LanguageSelect.js do zmiany języka. Źródło: opracowanie własne.	19
Rysunek 6 Komenda do instalacji frameworku express. Źródło: https://expressjs.com/	20
Rysunek 7 Podstawowy plik index.js inicjalizujący serwer Express. Źródło: https://www.npmjs.com/package/express	21
Rysunek 8 Instalacja zależności Sequelize. Źródło: https://sequelize.org/	22
Rysunek 9 Model User z zdefiniowanymi parametrami. Źródło: opracowanie własne.....	23
Rysunek 10 Zdefiniowane hooki do hashowania hasła. Źródło: opracowanie własne.	24
Rysunek 11 Funkcja checkPassword służąca do weryfikacji poprawności podanego hasła. Źródło: opracowanie własne.	24
Rysunek 12 Asocjacje modelu User z innymi encjami w systemie. Źródło: opracowanie własne.....	25
Rysunek 13 Plik index.js odpowiedzialny za ładowanie modeli i tworzenie powiązań między nimi. Źródło: opracowanie własne.	26
Rysunek 14 Wywołanie db.sequelize.sync() inicjalizujące tabele w bazie danych na podstawie zdefiniowanych modeli. Źródło: opracowanie własne.	26
Rysunek 15 Komenda do instalacji narzędzia Cypress. Źródło:	27
Rysunek 16 Komenda do uruchomienia graficznego interfejsu Cypress. Źródło:	27
Rysunek 17 Komenda do instalacji frameworku Jest. Źródło: https://jestjs.io/docs/getting-started	28
Rysunek 18 Definicja skryptu umożliwiająca uruchamianie testów w pliku package.json. Źródło: https://jestjs.io/docs/getting-started	28
Rysunek 19 Diagram klas systemu. Źródło: opracowanie własne.....	30
Rysunek 20 Ekran logowania klienta z odnośnikami do logowania dla innych ról. Źródło: opracowanie własne.	31
Rysunek 21 Weryfikacja tokenu JWT. Źródło: opracowanie własne.	32
Rysunek 22 Ekran dodania produktu firmy. Źródło: opracowanie własne.....	33
Rysunek 23 Widok katalogu produktów z perspektywy klienta. Źródło: opracowanie własne.	34
Rysunek 24 Diagram stanów procesu zamówienia. Źródło: opracowanie własne.	34
Rysunek 25 Ekran dodania wydarzenia firmy. Źródło: opracowanie własne.	35
Rysunek 26 Ekran zapisu na wydarzenie. Źródło: opracowanie własne.	36
Rysunek 27 Diagram stanów wydarzeń. Źródło: opracowanie własne	37

Rysunek 28 Ekran dodania artykułu. Źródło: opracowanie własne.	38
Rysunek 29 Widok artykułu wraz z komentarzami. Źródło: opracowanie własne.	39
Rysunek 30 Panel administracyjny do zarządzania klientami. Źródło: opracowanie własne.	40
Rysunek 31 Konfiguracja połączenia z testową bazą danych. Źródło: opracowanie własne..	41
Rysunek 32 Fragment kodu odpowiedzialny za przełączenie środowiska na testowe. Źródło: opracowanie własne.	42
Rysunek 33 Wyniki testów scenariusza rejestrowania. Źródło: opracowanie własne.	46
Rysunek 34 Wyniki testów scenariusza logowania. Źródło: opracowanie własne.	48
Rysunek 35 Wyniki testów walidacji tokenów JWT. Źródło: opracowanie własne.....	49
Rysunek 36 Wyniki testów edycji danych konta. Źródło: opracowanie własne.....	51
Rysunek 37 Wyniki testów operacji PUT. Źródło: opracowanie własne.	52
Rysunek 38 Wyniki testów operacji GET. Źródło: opracowanie własne	53
Rysunek 39 Wyniki testów integracyjnych. Źródło: opracowanie własne.....	57
Rysunek 40 Wyniki testów operacji POST. Źródło: opracowanie własne	58
Rysunek 41 Wyniki testów operacji DELETE. Źródło: opracowanie własne	59
Rysunek 42 Podsumowanie wyników przeprowadzonych testów. Źródło: opracowanie własne.....	59
Rysunek 43 Wyniki testów scenariuszy rejestrowania - frontend. Źródło: opracowanie własne.	62
Rysunek 44 Wyniki testów scenariuszy logowania - frontend. Źródło: opracowanie własne.	63
Rysunek 45 Wyniki testów scenariuszy dodawania wydarzenia. Źródło: opracowanie własne.	66
Rysunek 46 Wyniki testu scenariusza zapisu na wydarzenie. Źródło: opracowanie własne..	67
Rysunek 47 Wyniki testu scenariuszy dodania artykułu. Źródło: opracowanie własne.....	68
Rysunek 48 Wyniki testu scenariusza komentowania artykułu. Źródło: opracowanie własne.	69
Rysunek 49 Aktualna obsługa zasobów graficznych. Źródło: opracowanie własne.	70

Spis tabel

Tabela 1 Scenariusze testowe rejestrowania - backend	42
Tabela 2 Scenariusze testowe logowanie - backend	46
Tabela 3 Scenariusze testowe tokenów JWT	48
Tabela 4 Scenariusze testowe edycji profilu	50
Tabela 5 Scenariusze testowe przykładowe operacje PUT	51
Tabela 6 Scenariusze testowe przykładowych operacje GET	52
Tabela 7 Scenariusze testowe integracji modułów	53
Tabela 8 Scenariusze testowe przykładowe operacje POST	57
Tabela 9 Scenariusze testowe operacji DELETE	58
Tabela 10 Scenariusze testowe logowania - frontend	59
Tabela 11 Scenariusze testowe logowania - frontend	62
Tabela 12 Scenariusze testowe dodawania wydarzenia	63
Tabela 13 Scenariusze testowe udziału w wydarzeniu	66
Tabela 14 Scenariusze testowe dodania artykułu	67
Tabela 15 Scenariusze testowe komentowania artykułu	68